SQL-LINQ forráskód fordító dokumentáció

Ivádi Zsombor

2026

Tartalomjegyzék

1. Bevezetés **3**

2. A probléma bemutatása **4**
2.1. Az objektum-relációs impedancia-eltérés 4
2.2. Az adatbázis-elérés megközelítései 4
2.3. Az SQL és LINQ típusrendszerének kapcsolata 5
2.4. A manuális átírás problémái 6

3. Szakirodalmi áttekintés **7**
3.1. Fordítóprogram-eszközök és az ANTLR 7
3.2. Objektum-relációs impedancia-eltérés 8
3.3. LINQ és az Entity Framework Core 9
3.4. A LINQ alkalmazása relációs adatbázisokon túl 10
3.5. Roslyn és kódgenerálás 11
3.6. Transpilerek az iparban 12

4. Megvalósítás **12**
4.1. A tervezési döntések háttere 12
4.2. A grammatika és a parser 13
4.3. Az absztrakt szintaxisfa 18
4.4. A SELECT fordítása 20
4.4.1. A visitor és a scope kezelés 20
4.4.2. Kódgenerálás 21
4.4.3. Főbb kihívások 22
4.5. A DELETE fordítása 23
4.6. Az INSERT fordítása 25
4.6.1. VALUES alapú INSERT 25
4.6.2. INSERT INTO ... SELECT 26
4.7. Az UPDATE fordítása 26

5. Tesztelés **28**
5.1. Transpiler tesztek 29
5.2. Szemantikai ekvivalencia tesztek 30
5.3. Fuzz tesztek 31

6. LLM összehasonlítás **32**
6.1. Módszertan 32
6.2. Eredmények 33
6.3. Megfigyelések 33
6.4. Következtetések 34

7. A kódgenerálás leválasztása **35**
7.1. Motiváció és kiinduló állapot 35
7.2. Lehetséges megközelítések 36
7.2.1. XSLT alapú transzformáció 36
7.2.2. Deklaratív szabályrendszer 37
7.2.3. Kódgenerátor interfész 37
7.3. Az ICodeGenerator megvalósítása 38
7.4. Az átalakítás ellenőrzése 41
7.5. Korlátok 41

8. Összegzés **42**

1. Bevezetés

A modern szoftverrendszerek jelentős része valamilyen adatbázis-kezelő rendszert használ az adatok tárolására és lekérdezésére. Az adatbázisok kezelésének egyik legelterjedtebb eszköze az SQL (Structured Query Language), amely deklaratív módon teszi lehetővé az adatok lekérdezését és módosítását. Az SQL széles körben alkalmazott technológia, amely mára az adatbázis-kezelő rendszerek alapvető lekérdezési nyelve.

A modern programozási környezetekben azonban gyakran szükség van arra, hogy az adatbázis-lekérdezések szorosan illeszkedjenek az alkalmazás forráskódjához. Erre a problémára különböző programozási nyelvek eltérő megoldásokat kínálnak. A Java környezetben például a Stream API biztosít lehetőséget kollekciók deklaratív feldolgozására, míg a funkcionális programozási nyelvekben, mint például a Scala vagy a Kotlin, a beépített kollekció-műveletek hasonló szemléletet követnek. Ezek a megoldások lehetővé teszik az adatok feldolgozásának kifejezését deklaratív módon.

A .NET környezetben erre a problémára kínál megoldást a Language INtegrated Query (LINQ), amely a lekérdezési lehetőségeket közvetlenül a programozási nyelv szintaxisába integrálja. A LINQ lehetővé teszi, hogy a fejlesztők egységes módon kezeljék a különböző adatforrásokból származó adatokat, legyen szó objektumgyűjteményekről, adatbázisokról vagy XML dokumentumokról. Ennek köszönhetően az alkalmazás üzleti logikája és az adatlekérdezések ugyanabban a programozási környezetben valósíthatók meg [20, 5]. A .NET ökoszisztémában az Entity Framework Core az egyik legelterjedtebb ORM keretrendszer, amely a LINQ lekérdezéseket közvetlenül SQL utasításokká fordítja, így a fordító célplatformjaként kézenfekvő választásnak bizonyult.

A különböző megközelítések közül jelen dokumentáció a LINQ alapú lekérdezések feldolgozásával és generálásával foglalkozik, mivel a .NET ökoszisztémában széles körben alkalmazott megoldást jelent, valamint jól vizsgálható rajta keresztül az SQL és a magasabb szintű programozási nyelvek közötti leképezés problémaköre.

A dokumentáció egy olyan fordító (transpiler) tervezését és megvalósítását mutatja be, amely SQL lekérdezések automatikus LINQ kifejezésekké alakítására képes [4]. A projekt célja annak vizsgálata, hogy milyen módon valósítható meg egy ilyen fordító, milyen korlátok merülnek fel az SQL és LINQ közötti leképezés során, valamint milyen architekturális megoldások szükségesek egy ilyen rendszer létrehozásához.

Nagy nyelvi modellek (LLM) szintén képesek SQL lekérdezéseket LINQ kóddá alakítani, azonban ez a folyamat statisztikai alapú. Ugyanarra a bemenetre eltérő kimenet keletkezhet, a helyesség nem garantált, és az eredmény

nem integrálható automatizált pipeline-ba. Egy determinisztikus, formálisan definiált grammatikán alapuló fordító ezzel szemben verifikálható, reprodukálható és programozottan hívható. Ezt a megközelítést vizsgálja a jelen munka.

A megvalósítás során külön figyelmet kap a lekérdezések feldolgozásának folyamata, az absztrakt szintaxisfa (Abstract Syntax Tree) alapú reprezentáció kialakítása, valamint a generált LINQ kód előállítása [19].

2. A probléma bemutatása

2.1. Az objektum-relációs impedancia-eltérés

A szoftverfejlesztés során az egyik leggyakrabban felmerülő architekturális kihívás az objektum-relációs impedancia-eltérés (object-relational impedance mismatch). Ez a jelenség abból fakad, hogy a relációs adatbázisok és az objektumorientált programozási nyelvek alapvetően eltérő paradigmákra épülnek. Míg az SQL alapú rendszerek adathalmazokban, táblákban és sorokban tárolják az információt, addig a C# és más modern objektumorientált nyelvek objektumokban, valamint memóriabeli referenciagráfokban gondolkodnak [20].

Az SQL egy tisztán deklaratív nyelv, amely arra fókuszál, hogy *mit* akarunk lekérdezni, a végrehajtás pontos lépéseinek definiálása nélkül. Bár a LINQ bevezeti a deklaratív adatelérés kényelmét a C# nyelvbe, alapvetően mégis egy típusbiztos, imperatív ökoszisztémába ágyazódik be, ahol a lekérdezések láncolt metódushívásokra épülnek.

2.2. Az adatbázis-elérés megközelítései

Az adatbázis-elérés különböző módszereit az 1. táblázat foglalja össze, az egyszerűbbtől a magasabb szintű absztrakció felé haladva:

Ezen különbségek áthidalására a fejlesztők a múltban gyakran nyers SQL lekérdezéseket ágyaztak be az alkalmazás forráskódjába stringek formájában. Ez a megközelítés azonban komoly karbantarthatósági és biztonsági problémákkal járt. A beágyazott SQL stringek esetében a fordítóprogram (compiler) nem képes fordítási idejű (compile-time) szintaktikai vagy szemantikai ellenőrzést végezni. Ebből adódóan egy elgépelt oszlopnév vagy egy típusillesztési hiba kizárólag futásidőben (runtime) derül ki, ami jelentősen növeli a szoftver meghibásodásának kockázatát. Emellett az adatbázis-séma változásakor (például egy mező átnevezésekor) a modern integrált fejlesztőkörnyezetek (IDE) refaktoráló eszközei nem képesek automatikusan frissíteni a stringekben rejlő hivatkozásokat. Ezzel szemben a LINQ használatakor a

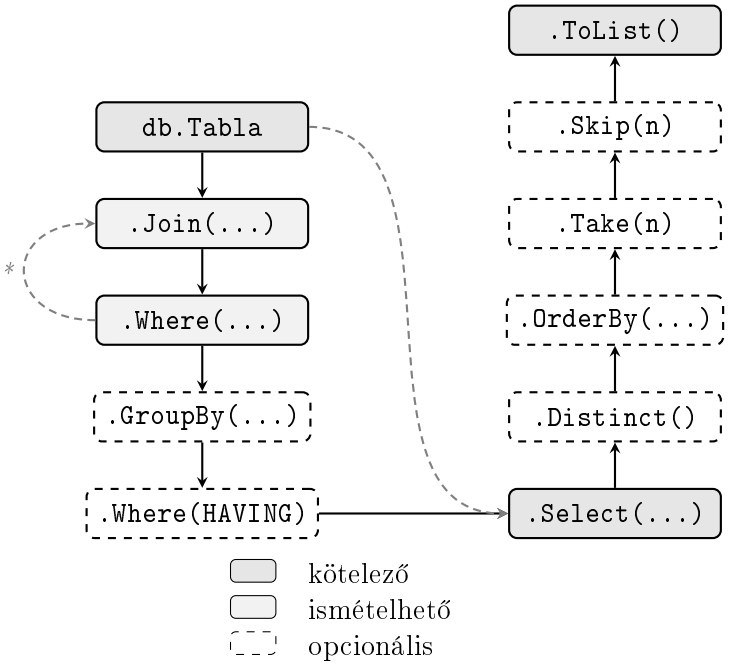
1. táblázat. Az adatbázis-elérés megközelítései

Megközelítés	**Jellemző**	**Példa**
Nyers SQL string	Fordítási idejű ellenőrzés nélkül, SQL injekció veszélyével	`"SELECT * FROM Users WHERE Id = " + id`
Paraméteres SQL	Biztonságos, de string alapú, sémaváltozásra érzékeny	`"SELECT * FROM Users WHERE Id = @id"`
Micro-ORM (Dapper)	SQL stringek erős típusmegfeleltetéssel	`conn.Query<User>( "SELECT ...", new { Id = id })`
LINQ + EF Core	Típusbiztos, fordítási idejű ellenőrzéssel, refaktorálható	`db.Users .Where(x => x.Id == id) .ToList()`

típusos modelleken végzett módosítások a teljes lekérdezés-láncot automatikusan és biztonságosan frissítik [20].

2.3. Az SQL és LINQ típusrendszerének kapcsolata

A LINQ kétféle szintaxisban írható fel. Lekérdezési szintaxisban (*query syntax*), amely az SQL-hez hasonló kulcsszavakat használ, és metódusszintaxisban (*method syntax*), amely láncolt metódushívásokból áll. A fordító az utóbbit generálja. A lekérdezési szintaxis olvashatóbb ugyan egyszerű esetekben, de a kódgenerálás szempontjából lényeges hátránya, hogy a záradékok egy egységes blokkban jelennek meg, ahol az opcionális elemek elhagyása külön kezelést igényel. A metódusszintaxisnál ezzel szemben minden záradék önálló metódushívás, amelyet a generátor egyszerűen kihagyhat, ha az adott SQL utasításban nem szerepel. Ez a szerkezet közvetlen megfelelést teremt az AST csomópontjai és a generált lánc tagjai között, ami a kódgenerálást szabályosabbá és bővíthetőbbé teszi.



1. ábra. A LINQ metódusszintaxis lánc általános alakja

2.4. A manuális átírás problémái

Nagyobb méretű legacy rendszerek modernizálása során gyakori feladat a több ezer soros, elavult adatbázis-hívások átírása modern, Entity Framework Core alapú LINQ kifejezésekké. Ennek manuális elvégzése rendkívül időigényes, monoton és hibalehetőségekkel teli folyamat. A két nyelv szintaxisa és operátor-készlete közötti rejtett különbségek könnyen nehezen felderíthető logikai hibákhoz vezethetnek. Szemléletes példa lehet erre a logikai operátorok precedenciájának eltérése. Míg az SQL-ben a `NOT` operátor egyértelműen a teljes relációra vonatkoztatható anélkül, hogy típuskonfliktust okozna, addig a C# `!` operátorának magasabb precedenciája miatt a kézi átírás során a megfelelő zárójelezés elhagyása azonnali fordítási hibát eredményezhet.

Ezen karbantartási és migrációs problémák kiküszöbölésére hatékony megoldást jelenthet egy transpiler megtervezése és alkalmazása. Egy ilyen eszköz létrehozása azonban specifikus technológiai kihívásokat is tartogat. Mivel a két nyelv nem feleltethető meg egymásnak egy az egyben, a fordítónak intelligens módon kell áthidalnia a hiányosságokat. Nem minden SQL utasításnak vagy beépített adatbázis függvénynek van közvetlen C# megfelelője. Például az SQL `LIKE` operátorának és helyettesítő karaktereinek fordítására kézenfekvő megoldás lenne reguláris kifejezések generálása lenne, ez a megoldás azonban kliensoldali kiértékeléshez vezet. Továbbá a fordító által előállított

LINQ kódnak nem csupán szintaktikailag kell helyesnek lennie, hanem illeszkednie kell az Entity Framework kifejezésfa (Expression Tree) kiértékelési mechanizmusaihoz is, elkerülve a nem támogatott szerveroldali fordításokat és a teljesítményt rontó kliensoldali adatfeldolgozást [4].

A bemutatott fordító célja az SQL:1999 szabvány által definiált lekérdezésnyelv egy jól meghatározott részhalmazának lefordítása Entity Framework Core 8.0 kompatibilis LINQ metódusszintaxisra. A bemeneti nyelv az SQL:1999-es szabványt követi, a generált kimenet pedig olyan C# kód, amely EF Core 8.0 alatt, bármely adatbázis-provider esetén szerver oldalon fordítható SQL-re.

A fordító hatóköre tudatosan a lekérdező és adatmanipulációs utasításokra korlátozódik, vagyis a `SELECT`, `INSERT`, `UPDATE` és `DELETE` műveletekre. Az SQL:1999 szabvány ezen felül adatdefiníciós (DDL) utasításokat is definiál, mint a `CREATE TABLE`, az `ALTER TABLE` vagy a `DROP TABLE`. Ezeknek azonban nincs LINQ megfelelőjük. Ennek oka, hogy az Entity Framework Core a sémát nem lekérdezésekkel kezeli, hanem a C# entitásosztályokból és a `DbContext` konfigurációjából vezeti le. Egy `CREATE TABLE` utasítás fordításának eredménye tehát nem egy LINQ kifejezés lenne, hanem egy entitásosztály és a hozzá tartozó modellkonfiguráció. Ezt a feladatot az EF Core már lefedi, hiszen a meglévő adatbázisból képes entitásosztályokat és `DbContext`-et előállítani. A grammatika ennek megfelelően a legfelső szintű `query` szabályban csak a négy támogatott utasítástípust ismeri, így egy DDL utasítás már a szintaktikai elemzés során kivételt vált ki, és nem jut el a visitor rétegig.

3. Szakirodalmi áttekintés

A fordító több egymástól eltérő, de szorosan összefüggő kutatási területre támaszkodik: a fordítóprogram-elméletre és az ehhez kapcsolódó eszközökre, az objektum-relációs leképezés problémakörére, a LINQ és az Entity Framework Core belső működésére, valamint a transpilerek ipari alkalmazásaira.

3.1. Fordítóprogram-eszközök és az ANTLR

A fordító megvalósításának alapját Terence Parr *The Definitive ANTLR 4 Reference* [19] című munkája adja. Ez az átfogó referenciamű tárgyalja az ANTLR 4 parser generátor működését, a kontextusfüggetlen grammatikák írásának módszertanát, a lexer és parser automatikus előállítását, valamint a parse fa bejárásának két fő mintáját, a listener- és a visitor-alapú megközelítést. Az ANTLR 4 egyik legfontosabb újítása a korábbi verziókhoz képest az Adaptive LL(*) (ALL*) algoritmus, amely a hagyományos LL(k) elemzőknél

rugalmasabban kezeli a grammatikai kétértelműségeket és nem igényel balrekurzió eltávolítást. A projekt szempontjából különösen releváns a könyv visitor mintával foglalkozó fejezete, mivel a projekt látogató (visitor) alapú megközelítést használ. A látogató lehetővé teszi, hogy a parse fa bejárásakor minden csomóponttípushoz külön feldolgozási logikát rendeljünk, és a visszatérési értéket felfelé propagáljuk a fában. Ezt a mechanizmust alkalmazza a fordító `SqlVisitor` osztálya is.

Parr korábbi, *Language Implementation Patterns* [18] munkája tágabb elméleti kontextusba helyezi az ANTLR-t és általában a nyelvfeldolgozás tervezési mintáit. A könyv külön mintaként tárgyalja a külső fa-látogatót (external tree visitor), amelynek lényege, hogy az AST csomópontjain végzett műveletek logikája elkülönül magától a fastruktúrától. Ez a szétválasztás lehetővé teszi, hogy ugyanazon szintaxisfát különböző célokra járjuk be anélkül, hogy a csomópontok definícióját módosítani kellene. A projekt is ezt az elvet követ. Az ANTLR által generált parse fán a `SqlVisitor` halad végig, és egy saját, a LINQ kódgeneráláshoz optimalizált absztrakt szintaxisfát (`LinqNode` hierarchiát) épít fel belőle. Ez utóbbi aztán a `ToCodeString()` metóduson keresztül állítja elő a végleges C# forráskódot.

Ortin és társai [15] empirikus összehasonlítást végeztek az ANTLR és a hagyományos Lex/Yacc eszközpáros között. Eredményeik szerint az ANTLR szignifikánsan alacsonyabb fejlesztési időt és hibaarányt eredményez, különösen összetett, több alternatívát tartalmazó grammatikák esetén. Ez megerősíti az ANTLR választásának indokoltságát a projektben, ahol a teljes SQL `SELECT` szintaxis lefedése volt a cél.

3.2. Objektum-relációs impedancia-eltérés

Az objektum-relációs impedancia-eltérés (object-relational impedance mismatch, ORIM) problémáját a szakirodalom régóta ismeri. Neward ezt a szembenállást találóan a számítástechnika Vietnámjaként jellemezte [14], arra utalva, hogy az összekapcsolás látszólag egyszerű feladatnak tűnik, ám a megvalósítás során egyre mélyebb és nehezebben feloldható ellentmondások bukkannak fel. Ireland és társai [8] strukturált keretrendszert dolgoztak ki az eltérések osztályozására, és négy fő kategóriát azonosítottak: adatmodell-eltérés (az objektumgráf és a relációs táblaszerkezet között), sémabeli eltérés (az öröklés és polimorfizmus vs. egyforma sorok), adatmanipulációs eltérés (az OOP metódushívásai vs. a halmazorientált SQL), valamint tranzakciós eltérés (az objektum életciklusa vs. a tranzakció határai). A fordító tervezésekor mindegyik kategória konkrét megvalósítási kihívásként jelentkezett. Például az SQL `NULL` kezelése alapvetően eltér a C# nullable típusaitól, az SQL aggregátumok csoportosított adathalmazon értelmezettek, míg a LINQ

aggregátumok az `IEnumerable` interfészen dolgoznak.

Papaioannou és Katsanos [17] 2020-as empirikus vizsgálata az ORIM hatásait mérte ORM keretrendszerek teljesítményére. Eredményeik rávilágítanak arra, hogy az ORM által generált SQL lekérdezések egyes esetekben lényegesen kevésbé hatékonyak a kézzel írott megfelelőiknél, különösen összetett JOIN műveleteket és aggregációkat tartalmazó lekérdezéseknél. Ez a megfigyelés relevanciával bír a projekt egyik tervezési döntésénél, mivel a transpiler által generált LINQ kódnak nemcsak szintaktikailag kell helyesnek lennie, hanem az Entity Framework Core query provider számára szerver oldalon lefordíthatónak is, elkerülve a teljesítményt rontó kliensoldali kiértékelést.

3.3. LINQ és az Entity Framework Core

A LINQ elméleti és gyakorlati hátterét Jon Skeet *C# in Depth* [20] munkája tárgyalja részletesen. A könyv különös figyelmet szentel a lusta kiértékelés (deferred execution) mechanizmusának, a kifejezésfa (expression tree) és az `IQueryable<T>` interfész kapcsolatának. Amíg a lekérdezés `IQueryable<T>` formában él, az EF Core képes azt kifejezésfaként elemezni és SQL-re fordítani. Amint azonban `IEnumerable<T>`-vé válik (pl. egy közbülső `ToList()` hívás következtében), az EF Core elveszti a SQL-generálás lehetőségét, és a kiértékelés C# szinten, a memóriában folytatódik. A könyv továbbá hangsúlyozza, hogy a LINQ metódusszintaxisa (method syntax) és lekérdezési szintaxisa (query syntax) szemantikailag ekvivalens, azonban az előbbi programozottan könnyebben előállítható.

Freeman és Rattz [5] könyve a LINQ teljes spektrumát lefedi. A projekt szempontjából különösen értékes a JOIN operátorok részletes tárgyalása. A `GroupJoin + SelectMany(DefaultIfEmpty)` kombináció mint a `LEFT OUTER JOIN` LINQ-beli megfelelője, az equi-join (.Join()) és a nem egyenlőség alapú (SelectMany + Where) JOIN-ok különbségei, valamint a több táblát érintő join-láncok felépítése. Ezek az ismeretek közvetlenül hasznosultak a fordító JOIN-kezelési logikájának tervezésekor.

A Microsoft hivatalos EF Core dokumentációja [12] részletezi, hogy az EF Core provider egyes LINQ operátorokat (mint például a `GroupJoin`) nem minden adatbázis-provider esetén képes szerver oldalon SQL-re fordítani. Ez a korlát a projekt tesztelési infrastruktúrájában is konkrét kihívásként jelentkezett, mivel az SQLite provider nem fordítja szerver oldalra a `GroupJoin` alapú LEFT/RIGHT JOIN mintákat, ami azt jelenti, hogy a lánc kliensoldali LINQ-to-Objects módban fut le, ahol a `DefaultIfEmpty()` ténylegesen `null` objektumot szúr be, és az azt követő tulajdonság-hozzáférések `NullReferenceException`-t dobhatnak. Az outer join szemantikai tesztelésére ezért egy dedikált, null-mentes seed adatokkal dolgozó adatbázis-sémán

történik.

3.4. A LINQ alkalmazása relációs adatbázisokon túl

A LINQ tervezésének kiindulópontját Meijer, Beckman és Bierman [9] munkája mutatja be. A szerzők célja egy olyan egységes lekérdezési modell kialakítása volt, amely az objektumgyűjtemények, a relációs adatok és az XML dokumentumok között nem tesz különbséget a programozó számára. Ezt a szabványos lekérdezőoperátorok és a cserélhető provider modell együttese teszi lehetővé. Ugyanaz a `Where`, `Select` vagy `OrderBy` hívás a memóriában tárolt gyűjteményen közvetlenül fut le, egy `IQueryable<T>` forrás esetén viszont kifejezésfaként jut el a providerhez, amely azt a saját célnyelvére fordítja. Ennek következtében a LINQ nem kötődik a relációs adatbázisokhoz, és több, egymástól lényegesen eltérő adatmodell lekérdezésére is alkalmazzák.

Az XML dokumentumok lekérdezésének hagyományos eszközei az XPath és az XQuery. A Microsoft dokumentációja [10] részletesen összeveti az XPath-t a LINQ to XML-lel, és a különbségek nagyrészt megegyeznek a 1. táblázatban bemutatott, SQL stringek és LINQ közötti különbségekkel. Az XPath kifejezés egy string, amelyet a fordítóprogram nem elemez, így a hibák csak futásidőben derülnek ki, az eredmény pedig gyakran típus nélküli objektum, amelyet a fejlesztőnek kell a megfelelő típusra alakítania. A LINQ to XML lekérdezéseit ezzel szemben a C# fordító ellenőrzi, a vetítések típusosak, és a lekérdezés eredménye nem csak a fa csomópontjaiból állhat, hanem tetszőleges új objektumból vagy XML szerkezetből is. A dokumentáció az XPath tengelyek többségéhez közvetlen LINQ to XML megfelelőt ad meg (a `child` tengelynek például az `Elements()`, a `descendant` tengelynek a `Descendants()` metódus felel meg), a `following` és `preceding` tengelyeknek azonban nincs közvetlen megfelelőjük. Ez a helyzet a jelen munkában vizsgált leképezéshez hasonló, ahol szintén a forrásnyelv egy jól körülhatárolható részhalmaza fordítható közvetlenül.

A dokumentumorientált adatbázisok területén a MongoDB 2024 májusában tette általánosan elérhetővé saját Entity Framework Core providerét, amely az EF Core 8 és a .NET 8 platformot támogatja [16]. A provider a code-first megközelítést, az alapvető CRUD műveleteket, a beágyazott dokumentumokat és a változáskövetést kezeli, a lekérdezések oldalán pedig a `Where`, `OrderBy` és `ThenBy` operátorokat, valamint az aggregátumokat fordítja le. A provider dokumentációja ugyanakkor számos korlátot is felsorol [13]. Nem támogatottak a migrációk és az idegen kulcsok, és a jelenlegi változat nem kezeli az eredményt átalakító `Select` vetítéseket sem, beleértve a névtelen típusokba történő vetítést. A projekt szempontjából ez a korlát különösen tanulságos, mivel a fordító a többoszlopos `SELECT` utasításokat ép-

pen névtelen objektumot előállító `Select` hívásra képezi le. A generált kód tehát szintaktikailag helyes maradna, ezen a provideren mégsem lenne futtatható. Ez jól szemlélteti, hogy egy LINQ kifejezés futtathatósága nem csak a nyelvtől, hanem a mögötte álló providertől is függ.

A HTML dokumentumok feldolgozására az AngleSharp .NET könyvtár kínál LINQ alapú megoldást [1]. A könyvtár a W3C szabványnak megfelelő DOM-ot valósít meg, amelyben a csomópontok a böngészőkből ismert `QuerySelector` és `QuerySelectorAll` metódusokkal, CSS szelektorok segítségével kereshetők, a DOM összes gyűjteménye pedig LINQ lekérdezésekkel tovább feldolgozható. A szelektorral kiválasztott elemekre így közvetlenül láncolható egy `Where` szűrés vagy egy `Select` vetítés. Az AngleSharp esetében a lekérdezés a LINQ to XML-hez hasonlóan a memóriában, LINQ to Objects módban fut le, és nincs olyan provider, amely a kifejezést más nyelvre fordítaná.

A bemutatott példák alapján a LINQ alkalmazásai két csoportba sorolhatók. Az első csoportban (LINQ to XML, AngleSharp) a lekérdezés a memóriában tárolt adatszerkezeten közvetlenül hajtódik végre, és a LINQ egy típusbiztos, a stringalapú lekérdezőnyelveket kiváltó API szerepét tölti be. A második csoportban (EF Core, MongoDB provider) a lekérdezés kifejezésfaként jut el a providerhez, amely azt egy másik lekérdezőnyelvre fordítja, így a provider képességei határozzák meg, hogy mely LINQ konstrukciók használhatók. A jelen munkában bemutatott fordító a második csoporthoz kapcsolódik, a vizsgált területek pedig további továbbfejlesztési irányokat jelölnek ki, például egy MongoDB providerhez igazodó vagy XPath kifejezéseket feldolgozó fordító formájában.

3.5. Roslyn és kódgenerálás

A jelen projekt tesztelési infrastruktúrájában a Microsoft Roslyn fordítóplatform is szerepet kapott. Vasani [21] és Harrison [7] munkái részletesen tárgyalják a Roslyn *Compiler as a Service* megközelítést, amelynek keretében a C# fordítófolyamat programozottan érhető el, lehetővé téve szintaktikai fák elemzését, szemantikai modellezést, kódgenerálást, valamint dinamikus kódfordítást és futtatást memóriában. A `FuzzTests` osztály a Roslyn `SyntaxFactory.ParseExpression()` metódusát alkalmazza a generált LINQ kifejezések szintaktikai érvényességének ellenőrzésére. Ez lehetővé teszi, hogy véletlenszerűen generált SQL lekérdezések LINQ-ra fordított kimenete szintaxishibák szempontjából anélkül legyen vizsgálható, hogy az érintett táblák valóban léteznének az adatbázisban.

3.6. Transpilerek az iparban

Az nMorph keretrendszer [4] egy transpiler-alapú megközelítést mutat be több programozási nyelvet érintő szoftverfejlesztési problémák megoldására. A keretrendszer rávilágít arra, hogy a transpiler-technológia nemcsak kódmigrációra alkalmazható, hanem általánosabb, többnyelvű szoftverfejlesztési munkafolyamatok automatizálásának eszközeként is.

Borne [2] esettanulmánya egy valódi ipari transpiler megvalósítását mutatja be, amely egy Oracle PL/SQL kódbázist fordít PostgreSQL-re. A cikk részletezi a megközelítés architektúráját, és hangsúlyozza, hogy az automatizált kódfordítás soha nem lehet teljes körű, mivel bizonyos konstrukciók, amelyek az egyik rendszerben természetesek, a másikban nem fejezhetők ki közvetlenül. Ez a megfigyelés összhangban van a projekt során szerzett tapasztalatokkal, mivel a SQL nyelv egyes elemei, például a `FULL OUTER JOIN`, a közös táblakifejezések (CTE) és az ablakfüggvények (`ROW_NUMBER`, `RANK`) nem rendelkeznek LINQ-ban olyan közvetlen megfelelővel, amely az Entity Framework Core által szerveroldali SQL-lekérdezéssé fordítható. Emiatt ezeknek a konstrukcióknak a támogatása a transpiler jelenlegi hatókörén kívül esik.

4. Megvalósítás

4.1. A tervezési döntések háttere

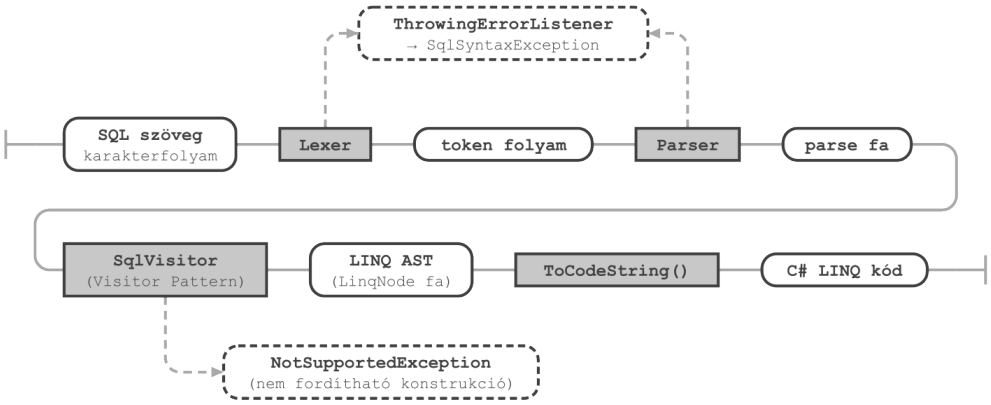
A fordító tervezésekor az első és legfontosabb kérdés az volt, hogy milyen szintű absztrakcióban érdemes gondolkodni. Az egyik szélsőséges megközelítés a közvetlen szöveges transzformáció lett volna, ahol a bemeneti SQL string reguláris kifejezések és szövegcserék útján alakul át LINQ kóddá. Ez a megközelítés egyszerűnek tűnik kis méretű, szabályos bemeneteken, de az SQL nyelv kontextusfüggő természete miatt gyorsan kezelhetetlenné válik. Egy feltétel például másképpen viselkedik `WHERE` záradékban, `HAVING` záradékban, allekérdezés belsejében vagy `CASE WHEN` kifejezésben, és ezeket a különbségeket reguláris kifejezéssel nem lehet megbízhatóan felismerni.

A másik véglet egy teljes fordítóprogram-lánc lett volna saját lexerrel, parserrel és szemantikai elemzővel. Ez ugyan a lehető legrugalmasabb megoldást adta volna, de az implementáció terhe aránytalanul nagy lett volna a projekt céljaihoz képest, és a karbantarthatóságot is megnehezítette volna.

A kettő között egy háromrétegű architektúra bizonyult a legjobb megoldásnak. Az első réteg egy ANTLR4-gyel generált parser, amely az SQL szöveget parse fává alakítja. A második réteg egy saját absztrakt szintaxisfa, amely a LINQ-specifikus struktúrákat reprezentálja. A harmadik réteg

a tényleges kódgenerátor, amely ebből az absztrakt reprezentációból állítja elő a végleges C# forráskódot. Ez a szétválasztás több előnnyel is jár. A grammatika és a kódgenerálás egymástól függetlenül fejleszthető és tesztelhető. Új SQL konstrukciók bevezetésekor elegendő egy új grammar szabályt, egy új AST-csomópontot és egy új visitor metódust hozzáadni, a rendszer többi része változatlan marad.

A fordítási folyamat lépéseit a 2. ábra szemlélteti.



2. ábra. A fordítási pipeline lépései

4.2. A grammatika és a parser

A parser előállítására az ANTLR4 parser generátor mellett döntöttem. Az ANTLR4 kontextusfüggetlen grammatikából automatikusan generál lexert és parsert, amelyek a bemeneti szöveget parse fává alakítják. A generált kód C# nyelvű, így közvetlenül integrálható a .NET alapú projektbe.

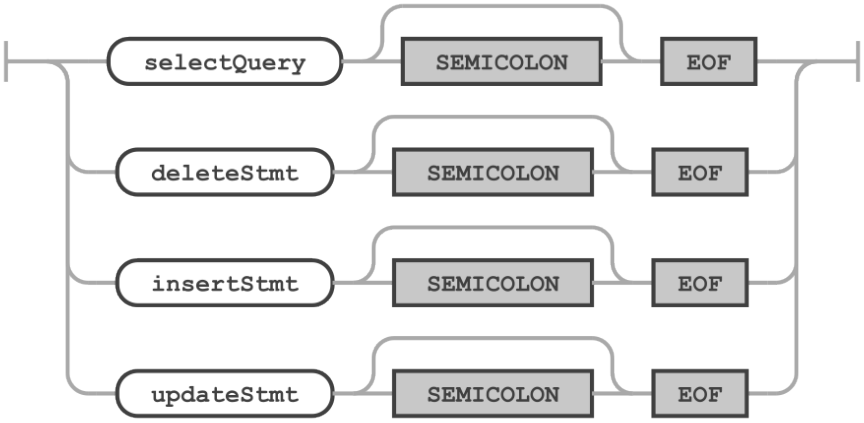
Az SQL grammatika megírása során fontos szempont volt, hogy a nyelvtan csak azt fedje le, amit a fordító valóban képes kezelni. Egy teljes SQL:1999-es grammatika hatalmas, és sok olyan konstrukciót tartalmaz amelynek LINQ-ban nincs közvetlen megfelelője. A projekt grammatikája ezért szándékosan az SQL nyelv egy szűkebb, de teljes egészében támogatott részhalmazát fedi le, amelynek minden szabályához rendelkezésre áll a megfelelő LINQ-kódgenerálási logika.

A lexer a bemeneti karakterfolyamból tokeneket állít elő. A kulcsszavak esetében figyelmen kívül hagyja a kis- és nagybetű különbséget, így a `SELECT`, `select` és `Select` alakok egyenértékűek. Az azonosítók felismerése után a fordító egy `ToPascalCase` konverziót végez, amely az SQL konvencióinak megfelelő `snake_case` vagy csupa nagybetűs neveket C#-ban szokásos

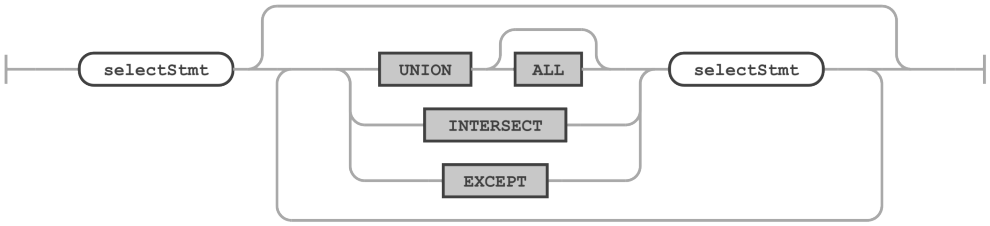
PascalCase formára hozza. Ez azt jelenti, hogy a `user_name` oszlopnévből automatikusan `UserName`, a `USERS` táblanévből pedig `Users` lesz.

A nyelvtan felépítése a legáltalánosabb szabályoktól halad a specifikusak felé. A legfelső szintű `query` szabály egy `selectQuery`-t vár, amelyet egy opcionális pontosvessző követhet. A `selectQuery` lehetőséget ad halmazműveletek láncolására (`UNION`, `INTERSECT`, `EXCEPT`), míg maga a `selectStmt` bontja ki az egyes lekérdezések szerkezetét. A `condition` szabály rekurzív, ami lehetővé teszi az `AND`, `OR` és `NOT` operátorok tetszőleges mélységű egymásba ágyazását. Az `expr` szabály szintén rekurzív, kezelve az aritmetikai kifejezéseket és a allekérdezéseket. Az ANTLR4 Adaptive LL(*) algoritmusának köszönhetően ezek a rekurzív szabályok kezelhetők anélkül, hogy kézzel kellene foglalkozni a kétértelműségekkel.

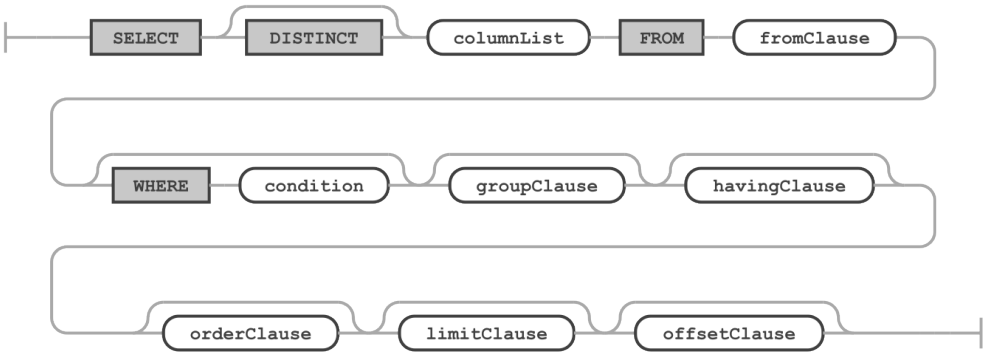
A főbb grammatikai szabályok railroad diagramjai a 3–8. ábrákon láthatók.



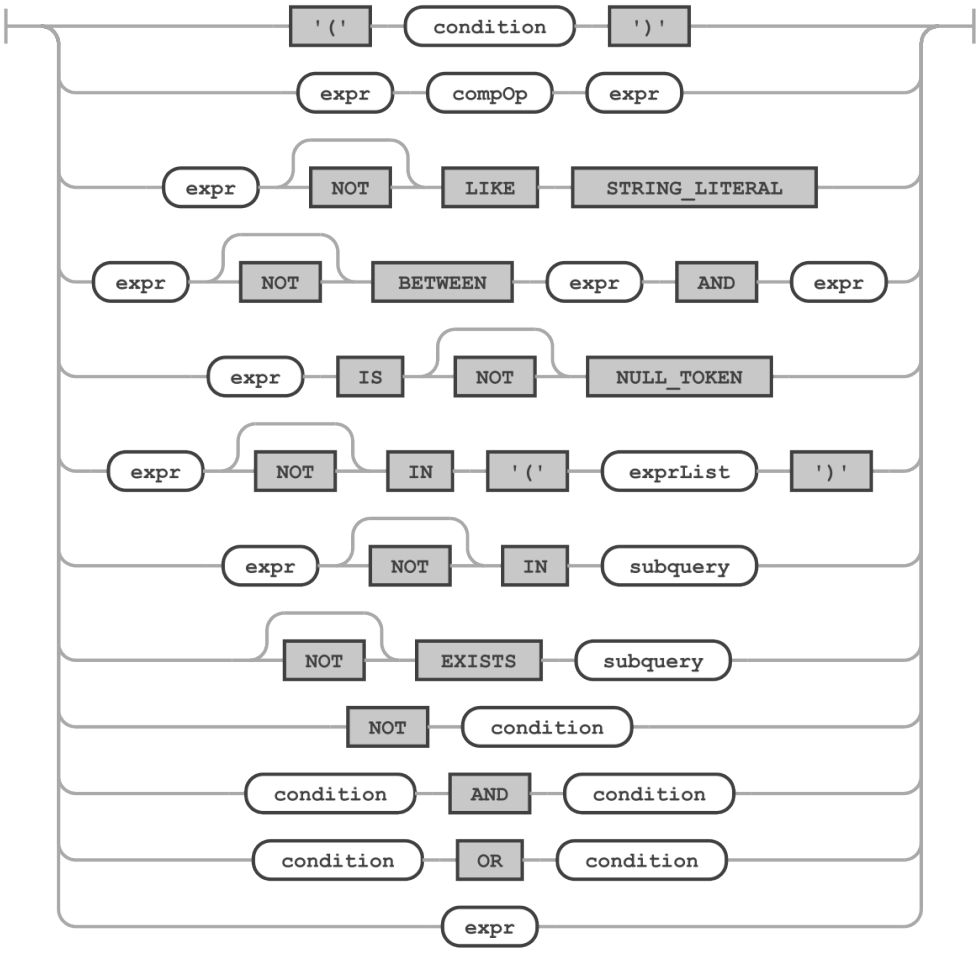
3. ábra. A `query` szabály railroad diagramja



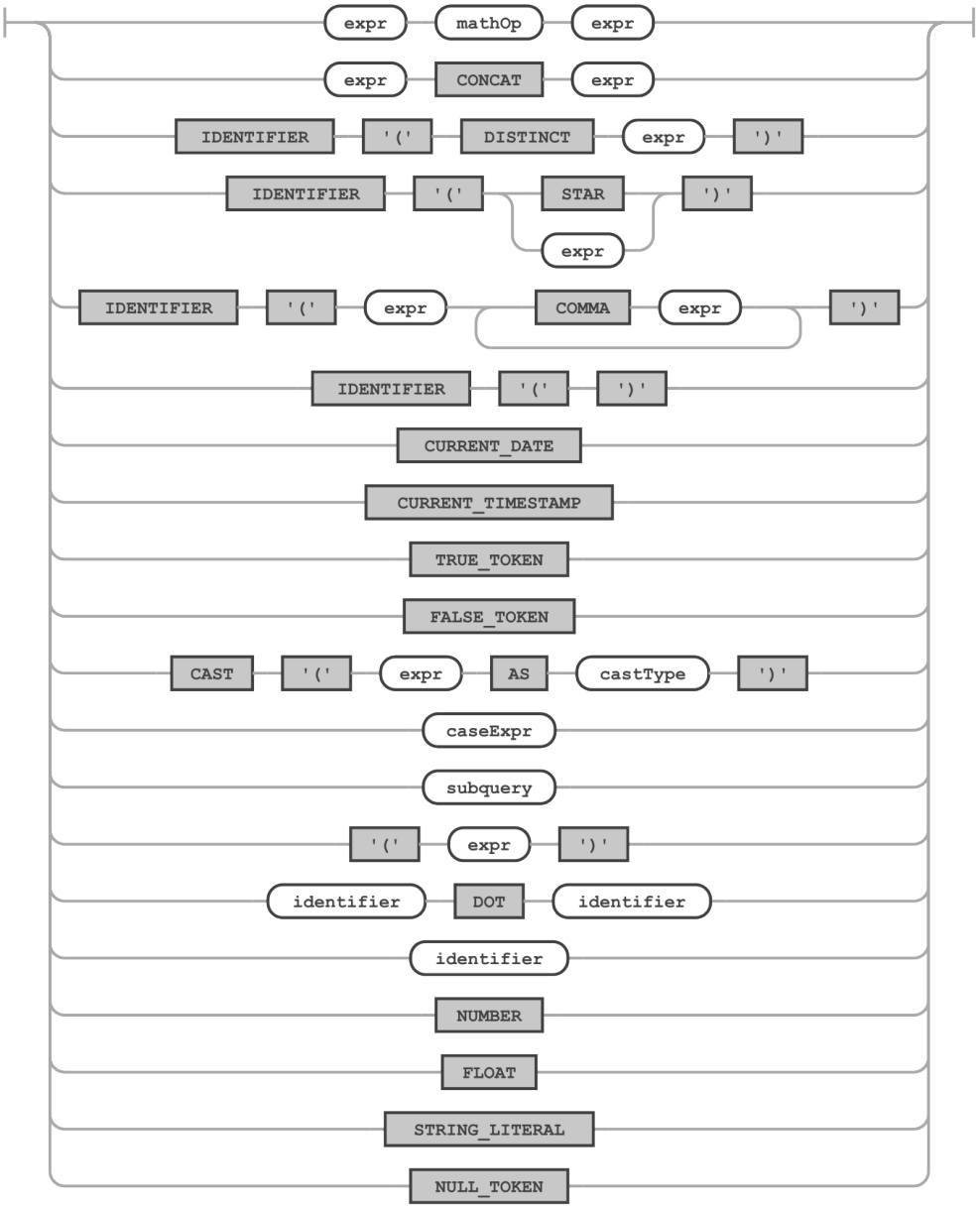
4. ábra. A `selectQuery` szabály railroad diagramja



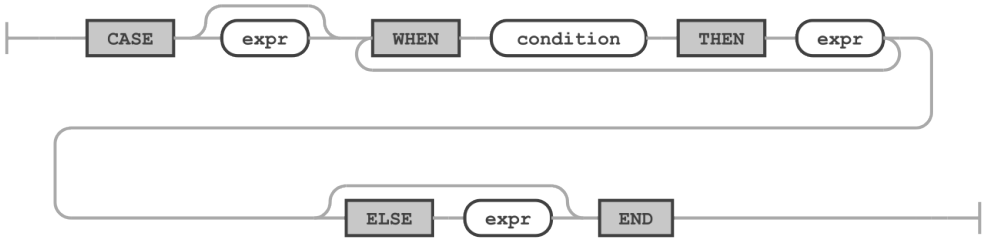
5. ábra. A `selectStmt` grammatikai szabály railroad diagramja



6. ábra. A `condition` grammatikai szabály railroad diagramja



7. ábra. Az `expr` grammatikai szabály railroad diagramja



8. ábra. A `caseExpr` grammatikai szabály railroad diagramja

A többargumentumú függvényhívások kezelésére a grammatika egy variadikus szabályt alkalmaz. Az első változatban a két-, három- és négyargumentumú esetekre külön-külön szabályok léteztek, amit egy általánosabb `funcCallExpr` szabály váltott fel, amely `expr (COMMA expr)+` alakban írható le. Ez a változtatás egyszerűsítette a nyelvtant és lehetővé tette, hogy az argumentumszám-érvényesítés a visitor rétegbe kerüljön, ahol értelmes hibaüzenettel jelezhető, ha egy függvényt helytelen számú paraméterrel hívnak meg.

4.3. Az absztrakt szintaxisfa

Az ANTLR4 által generált parse fa az SQL szintaxis közvetlen tükörképe, mivel minden grammatikai szabályhoz egy-egy csomópont tartozik. Ez a reprezentáció a kódgenerálás szempontjából túlságosan alacsony szintű, mert olyan részleteket tartalmaz amelyek a LINQ kód előállításához irrelevánsak, például a zárójeleket, a vesszőket és az SQL kulcsszavakat. Emiatt a fordító egy saját absztrakt szintaxisfát épít fel, amelynek csomópontjai már a LINQ struktúráknak megfelelő fogalmakat reprezentálnak.

A csomóponthierarchia alapja az absztrakt `LinqNode` osztály, amelynek egyetlen absztrakt metódusa a `ToCodeString()`, ez felelős a C# kód előállításáért. Minden konkrét csomóponttípus ebből öröklődik és saját implementációt ad ennek a metódusnak.

A `LinqQueryNode` tárolja a forrástábla nevét és a metódushívások rendezett listáját, ez képviseli az egész lekérdezést. A `LinqMethodCallNode` egyetlen LINQ metódushívást ír le, névvel és argumentumlistával.
A `LinqLambdaNode` egy lambda kifejezést tartalmaz paraméternévvel és törzzsel, és ez jelenik meg a `Where`, `Select`, `OrderBy` és hasonló metódusok belső kifejezéseként.
A `LinqAnonymousObjectNode` egy névtelen C# objektumot reprezentál, amelyet a többoszlopos `SELECT` vetítések állítanak elő. A `LinqJoinNode` a `JOIN`

konstrukciót modellezi, tárolja a join típusát, a belső tábla nevét, valamint a bal és jobb oldali kulcskifejezést.

A kifejezések szintjén a `LinqConstantNode` literális értékeket tárol, a `LinqIdentifierNode` azonosítókat és oszlophivatkozásokat, a `LinqBinaryExpressionNode` bináris kifejezéseket (aritmetika, összehasonlítás), a `LinqCaseNode` pedig a `CASE WHEN` szerkezetet modellezi, tárolja az opcionális operandust, a `WHEN` ágak listáját és az `ELSE` kifejezést.

A DML utasítások saját gyökércsomópontokat kaptak. A `LinqDeleteNode` a törlési utasítást reprezentálja a forrástábla névvel, az opcionális `WHERE` feltétellel és a lambda paraméternévvel. A `LinqInsertNode` a `VALUES` alapú beszúrást tárolja, soronként tartalmazza az oszlopnév-érték párokat. A `LinqInsertSelectNode` az `INSERT INTO ... SELECT` alakot kezeli, tárolja a céltáblát, az oszloplistát és a teljes SELECT lekérdezés AST-ját, amelyből kódgeneráláskor a projekciót újra kell írni. A `LinqUpdateNode` az `UPDATE` utasítást modellezi a `SET` értékadások listájával, amelynek minden eleme egy `(ColumnName, LinqNode)` pár.

Ez az elrendezés teszi lehetővé, hogy a kódgenerálás egyszerűen a csomópont-fa bejárásával valósuljon meg. Egy lekérdezés generálásához elegendő a gyökér `LinqQueryNode` `ToCodeString()` metódusát meghívni, amely rekurzívan meghívja a gyermek csomópontok ugyanezen metódusát, és az eredményeket összefűzi.

A LINQ kód egy lánc alakú metódushívás-sorozat, amelynek sorrendje kötött. A fordítónak ezt a sorrendet be kell tartania. Az AST-alapú megközelítés ezt természetesen kezeli, mivel a `LinqQueryNode` a metódusokat egy rendezett listában tárolja, és a visitor réteg gondoskodik arról, hogy ezek a helyes sorrendben kerüljenek bele.

Érdemes megfigyelni, hogy a két fa szerkezete lényegesen eltér egymástól. A parse fa szélességében nagy, egy egyszerű `SELECT Name FROM Users WHERE Age > 18` lekérdezésből több tucatnyi csomópontból álló fa keletkezik, amelynek levelein az egyes tokenek (a `SELECT` kulcsszó, az oszlopnév, a `FROM` kulcsszó, a táblanév stb.) találhatók. A LINQ AST ezzel szemben mélységében strukturált, ugyanebből a lekérdezésből egy `LinqQueryNode` keletkezik, amelynek `Methods` listájában egy `LinqMethodCallNode` (`Where`) és egy `LinqMethodCallNode` (`Select`) szerepel, és mindkettő egy-egy `LinqLambdaNode`-ot tartalmaz. Ez a szétválasztás teszi lehetővé, hogy a kódgenerálás triviálisan egyszerű legyen, mivel minden csomópont pontosan tudja, hogyan kell önmagát C# kóddá alakítani.

4.4. A SELECT fordítása

4.4.1. A visitor és a scope kezelés

A parse fa bejárására az ANTLR4 visitor mintáját alkalmazom. Az ANTLR4 a grammatikából automatikusan generálja az alap visitor interfészt, amelyet a `SqlVisitor` osztály implementál. Minden grammatikai szabályhoz egy `Visit...` metódus tartozik, amely a megfelelő `LinqNode` leszármazottat építi fel és adja vissza.

A visitor legösszetettebb része a scope kezelés, amelyre az SQL allekérdezések és korrelált hivatkozások miatt van szükség. Amikor egy belső `SELECT` lekérdezés hivatkozik a külső lekérdezés egy táblájára vagy aliasára, a fordítónak el kell döntenie, hogy az adott azonosítót melyik lambda paraméterrel kell prefixelni. Erre a célra az `_outerScopes` nevű verem szolgál, amelybe minden `SELECT` feldolgozásának kezdetén bekerül az aktuális tábla-alias leképezés és az aktuális lambda paraméter neve. Amikor a belső lekérdezés egy azonosítót nem talál meg a saját scope-jában, végigmegy a veremen és megkeresi azt a külső scope-ban, majd az ott tárolt lambda paramétert használja prefixként.

```
// Korrelált EXISTS allekérdezés generálása
// SQL: SELECT Name FROM Users u WHERE EXISTS
//        (SELECT 1 FROM Orders o WHERE o.Owner = u.Id)
// Generált LINQ:
db.Users.Where(x => (db.Orders.Where(o => o.Owner == x.Id
   )).Any())
         .Select(x => new { x.Name })
         .ToList()
```

9. ábra. Korrelált allekérdezés scope kezelése

A lambda paraméter nevének nyomon követése nem triviális. Az egyszerű, alias nélküli lekérdezésekben a lambda paraméter mindig `x`. Ha azonban a lekérdezésben explicit SQL alias szerepel, például `FROM Orders o`, a belső lambda paraméter neve az alias lesz, tehát `o`. Ez biztosítja, hogy a generált kód olvasható maradjon, és a korrelált hivatkozások egyértelműen azonosíthatóak legyenek a generált LINQ-ban.

A `GROUP BY` kontextus kezelése szintén különleges figyelmet igényel. A csoportosítás után a LINQ `Select` metódus lambda paraméterként a `g` csoportváltozót kapja, az oszlophivatkozások pedig `g.Key` vagy `g.FirstOrDefault().Oszlop` alakot öltenek. A visitor nyilvántartja, hogy

éppen `GROUP BY` kontextusban dolgozik-e, és ennek megfelelően módosítja a generált kifejezéseket. Egy belső allekérdezés látogatásakor a kontextus flagek mentésre kerülnek, és a belső lekérdezés feldolgozása után visszaállítódnak, hogy a külső lekérdezés kontextusa ne sérüljön.

A kód olvashatóságának és karbantarthatóságának javítása érdekében a fejlesztés során több ismétlődő minta kiszervezésre került újrafelhasználható segédmetódusokba. A `BuildWhereMethod` metódus elvégzi a `_inWhereClause` flag beállítását, a feltétel látogatását és a `LinqMethodCallNode` összerakását. Hasonlóan, a `BuildOrderByMethods` az `ORDER BY` záradék feldolgozását végzi, a `SetupTableScope` pedig a tábla-alias leképezés és a scope verem feltöltését. Ezek a segédmetódusok egységes belépési pontot biztosítanak, és lehetővé teszik, hogy a `DELETE` és az `UPDATE` utasítások is ugyanazokat a WHERE és ORDER BY generálási logikákat használják, amelyek a `SELECT` esetén implementálva lettek.

4.4.2. Kódgenerálás

A kódgenerálás a visitor által felépített AST-csomópontok `ToCodeString()` metódusainak rekurzív meghívásával valósul meg. A folyamat kiindulópontja mindig a gyökér `LinqQueryNode`, amelynek `ToCodeString()` metódusa a forrástábla nevével kezdi az eredményt (`db.Tabla`), majd sorban hozzáfűzi a `Methods` lista elemeinek string reprezentációját.

A többoszlopos `SELECT` esetén az `LinqAnonymousObjectNode` generál névtelen C# objektumot. Ha az oszlop neve megegyezik a forrásoszlop nevével, egyszerű shorthand-et alkalmaz (`x.Nev`), különben explicit hozzárendelést (`Alias = x.Nev`). A `SELECT *` speciális eset, ahol nem keletkezik `Select` metódushívás, csak a `ToList()` zárja a láncot.

Az allekérdezések generálása során fontos különbséget tenni a skaláris és a kollekcióalapú allekérdezések között. Az `IN (SELECT ...)` esetén a belső lekérdezés skaláris projekciót kap, és az összehasonlítás `Any()` metódussal történik, amely típusbiztos és nullable típusokat is helyesen kezel. Kivételt képez az az eset, amikor a belső lekérdezés maga is egy `FROM` záradékbeli allekérdezésre épül. Ilyenkor a belső lánc nem `LinqQueryNode`, hanem inline nézet, ezért a fordító a `Contains()` alapú alakra tér át. Az `EXISTS` esetén a belső lekérdezés vetítése irreleváns, ezért a fordító a `ToCodeStringUpToSelect()` segédmetódust hívja, amely a lánc `Select` metódusa elé áll meg, és a kapott lekérdezésen az `Any()` kerül meghívásra. A skaláris allekérdezések, mint például `(SELECT MAX(Age) FROM Users)` esetén a fordító ellenőrzi, hogy a belső lekérdezés már önmagában skaláris eredményt ad-e vissza (aggregátum hívás), és ennek megfelelően dönti el, hogy szükséges-e a `FirstOrDefault()` hozzáfűzése.

A `CASE WHEN` kifejezések beágyazott ternáris operátorokká alakulnak. Minden `WHEN` ágból egy `? : ` kifejezés keletkezik, amelyek egymásba ágyazódnak, az `ELSE` ág adja a legbelső `false` értéket. Ha nincs `ELSE` ág, a végső értékként `null` keletkezik, ami visszatükrözi az SQL szemantikáját.

A függvényleképezés a `LinqStringFunctionNode` osztályban egy `switch` kifejezéssel valósul meg. A matematikai függvények a `Math` osztály statikus metódusaira képződnek le, a stringfüggvények a `string` típus metódusaira, a dátumfüggvények pedig a `DateTime` struktúra property-jeire és metódusaira. Az ismeretlen függvénynevek esetén a `switch` alapértelmezett ága `NotSupportedException`-t dob, amely a fordítási folyamat során derül ki, nem futásidőben.

4.4.3. Főbb kihívások

A fejlesztés során több olyan pont adódott, ahol az első megközelítés tévesnek bizonyult és gondos átgondolást igényelt.

Az egyik ilyen kihívás a `LIKE` operátor kezelése volt. Az első implementáció reguláris kifejezésre fordította a `LIKE` mintákat: a `B%` mintából `(?i)^B.*$` regex lett, amelyet a generált kódban `Regex.IsMatch()` hívás használt fel. Ez C# szinten helyesen működött, de az Entity Framework Core nem képes a `Regex.IsMatch()` metódust SQL-re fordítani, ezért a kiértékelés kliensoldalon, a memóriában zajlott. Ez teljesítménybeli szempontból elfogadhatatlan lett volna éles környezetben, különösen nagy adathalmazok esetén. A megoldás az `EF.Functions.Like()` metódus alkalmazása volt, amely az EF Core által közvetlenül SQL `LIKE` utasítássá fordítható, és minden adatbázis-provider támogatja.

A korrelált allekérdezések scope kezelése szintén jelentős tervezési kihívást jelentett. A belső `SELECT` látogatásakor a visitor a saját scope-jában keresi az azonosítókat, és ha nem találja, át kell néznie a verem korábbi szintjeit. Az első implementáció egyszerűen elmentette az aktuális tábla-alias leképezést, majd a belső lekérdezés feldolgozása után visszaállította. Ez azonban nem volt elegendő, mert a belső lekérdezésen belüli lambda paraméter neve ütközhetett a külső lekérdezés lambda paraméterével, ami érvénytelen C# kódot eredményezett. A megoldás egy pontosabb scope modell bevezetése volt, amelyben a verembe kerülő bejegyzés nemcsak a tábla-alias leképezést, hanem az aktuális lambda paraméter nevét is tárolja, és a belső látogatás idején ezeket az értékeket elmentjük és visszaállítjuk.

A `GROUP BY` kontextusban megjelenő allekérdezések egy rejtettebb problémát okoztak. A `SELECT` lista látogatásakor a `GROUP BY` kontextus aktív, ami azt jelenti, hogy az oszlophivatkozások
`g.FirstOrDefault().Oszlop` alakra kell alakuljanak. Amikor azonban a

`SELECT` listában egy korrelált allekérdezés is szerepel, a belső lekérdezés látogatása felülírja az `_inWhereClause`, `_inAggregate` és `_inOrderBy` flag-eket, amelyek a külső lekérdezés kontextusát rögzítik. A belső lekérdezés befejezése után ezek a flag-ek nullázott állapotban maradtak, így a `SELECT` lista maradék elemei elveszítették a `GROUP BY` kontextust, és hibás lambda kifejezések keletkeztek. A megoldás egy egységes `SaveClauseState` és `RestoreClauseState` mechanizmus bevezetése volt, amely minden `SELECT` belépésekor elmenti, és kilépésekor visszaállítja a teljes kontextus-állapotot.

A nullable típusok kezelése az `IN (SELECT ...)` allekérdezéseknél szintén váratlan akadályt jelentett. Az első implementáció a belső lekérdezés által visszaadott szekvencián `Contains()` metódust hívott. Ez C# szinten és SQLite esetén helyesen működött, de amikor az összehasonlítandó értékek egyike nullable típusú volt, az EF Core SQL Server és PostgreSQL providerein típusillesztési hiba keletkezett, mivel az `IQueryable<int>` nem tartalmaz `Contains(int?)` metódust. A megoldás az `Any()` metódusra való áttérés volt, amely explicit egyenlőség-összehasonlítást végez, és típusbiztos módon kezeli a nullable típusokat minden provideren.

```
// Korábbi, típusillesztési hibát okozó megközelítés:
(db.Users.Where(u => u.Role == "Admin").Select(u => u.Id)
   )
    .Contains(x.Owner)   // CS1929: IQueryable<int> nem
       tartalmazza ezt

// Javított, provider-független megközelítés:
db.Users.Where(u => u.Role == "Admin")
        .Any(u => u.Id == x.Owner)
```

10. ábra. Nullable típusillesztési probléma és megoldása

4.5. A DELETE fordítása



11. ábra. A `deleteStmt` grammatikai szabály railroad diagramja

A `DELETE` utasítás fordítása az EF Core 8.0-ban bevezetett `ExecuteDeleteAsync()` metódusra épül. Ez a metódus közvetlenül az adat-

bázison hajt végre törlési műveletet, megkerülve az EF Core change trackert, ami azt jelenti, hogy nem szükséges az érintett entitásokat előbb a memóriába tölteni, majd törölni.

A fordítás alapesete a WHERE feltétel nélküli törlés, ahol a teljes tábla tartalma törlésre kerül.

```
// SQL: DELETE FROM Users
// Generált LINQ:
db.Users.ExecuteDeleteAsync()
```

12. ábra. DELETE WHERE feltétel nélkül

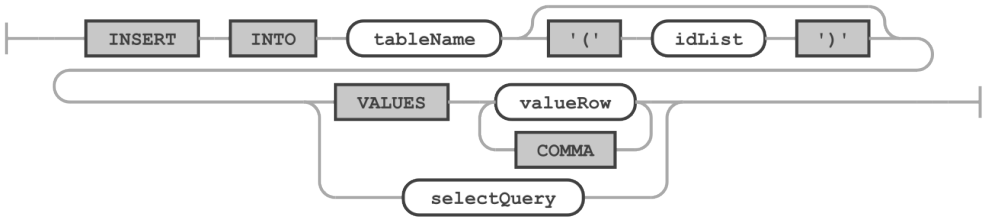
WHERE feltétel esetén a lánc egy `Where()` hívással egészül ki, amelybe a feltétel ugyanazzal a scope-kezelési mechanizmussal kerül, mint a `SELECT` esetén. Ez lehetővé teszi, hogy korrelált allekérdezések is megjelenjenek a `DELETE` feltételében.

```
// SQL: DELETE FROM Users WHERE EXISTS
//        (SELECT 1 FROM Orders o WHERE o.Owner = Users.
   Id)
// Generált LINQ:
db.Users.Where(x => (db.Orders.Where(o => o.Owner == x.Id
   )).Any())
        .ExecuteDeleteAsync()
```

13. ábra. DELETE korrelált allekérdezéssel

A `DELETE` implementációja a `SetupTableScope` és `BuildWhereMethod` segédmetódusokat újrahasználja, amelyek a `SELECT` feldolgozásához készültek. Ez jól szemlélteti a háromrétegű architektúra előnyét, mivel az új utasítástípus bevezetésekor a WHERE-kezelési logikát nem kellett újraírni.

4.6. Az INSERT fordítása



14. ábra. Az `insertStmt` grammatikai szabály railroad diagramja

Az `INSERT` utasítás két alakban támogatott. Az első a `VALUES` alapú beszúrás, a második az `INSERT INTO ... SELECT` alak, amely egy lekérdezés eredményét szúrja be a céltáblába.

4.6.1. VALUES alapú INSERT

Egyetlen sor beszúrásakor az `Add()` metódus generálódik, több sor esetén az `AddRange()` egy `List<T>` példánnyal. Mindkét esetben a generált kód egy `SaveChanges()` hívással zárul, amely az EF Core change trackeren keresztül véglegesíti a változásokat.

```
// SQL: INSERT INTO Users (Name, Age, Role) VALUES ('Bob
    ', 25, 'Admin')
// Generált LINQ:
db.Users.Add(new Users { Name = "Bob", Age = 25, Role = "
    Admin" });
db.SaveChanges();

// SQL: INSERT INTO Users (Name, Age) VALUES ('Bob', 25),
     ('Alice', 19)
// Generált LINQ:
db.Users.AddRange(new List<Users>
{
     new Users { Name = "Bob", Age = 25 },
     new Users { Name = "Alice", Age = 19 }
});
db.SaveChanges();
```

15. ábra. VALUES alapú INSERT fordítása

A `NULL` értékek kezelése egyszerű, a generált kódban a `null` kulcsszó jelenik meg az adott property értékeként, ami az EF Core számára egyértelműen adatbázis-szintű `NULL`-t jelent.

4.6.2. INSERT INTO ... SELECT

Ez az alak egy meglévő lekérdezés eredményét szúrja be a céltáblába. A fordítás során a SELECT lekérdezés AST-ja a `LinqInsertSelectNode`-ba kerül, amely a kódgeneráláskor a SELECT projekciót újraírja a céltábla típusára mutató inicializátorrá.

```
// SQL: INSERT INTO Orders (Owner, Item, Qty)
//      SELECT Id, 'Default', 0 FROM Users WHERE Role = '
   Admin'
// Generált LINQ:
db.Orders.AddRange(
    (db.Users.Where(x => x.Role == "Admin"))
        .Select(x => new Orders { Owner = x.Id, Item = "
           Default", Qty = 0 })
        .ToList()
);
db.SaveChanges();
```

16. ábra. INSERT INTO ... SELECT fordítása

A projekció újraírása AST szinten történik, nem szövegmanipulációval. A `LinqInsertSelectNode` a SELECT lánc végéről eltávolítja a névtelen objektumot előállító `Select()` hívást, és helyette egy typed inicializátort generál, amelynek property-jei az INSERT oszloplistájából, értékei pedig a SELECT projekció kifejezéseiből származnak. Ez biztosítja, hogy literálok (`'Default'`), konstansok (`0`) és oszlophivatkozások (`x.Id`) egyaránt helyesen kerüljenek be a generált kódba.

4.7. Az UPDATE fordítása



17. ábra. Az `updateStmt` grammatikai szabály railroad diagramja

Az `UPDATE` utasítás fordítása az EF Core 8.0-ban bevezetett `ExecuteUpdateAsync()` metódusra épül, amely a `ExecuteDeleteAsync()`-hoz hasonlóan közvetlenül az adatbázison hajt végre módosítást, a change tracker megkerülésével.

A `SET` záradék minden egyes értékadásából egy `.SetProperty()` hívás keletkezik, amelyek láncoltan kapcsolódnak egymáshoz. A `SetProperty()` két paramétert vár: az első egy lambda, amely azonosítja a módosítandó oszlopot, a második pedig az új értéket adja meg.

Az új érték megadásának alakja attól függ, hogy a jobb oldal tartalmaz-e oszlophivatkozást. Ha az értékadás jobb oldala kizárólag konstans (például `SET Role = 'Admin'`), a második argumentum direkt értékként jelenik meg. Ha azonban a jobb oldal hivatkozik az aktuális sorra (például `SET Points = Points + 10`), a második argumentum szintén lambda alakot ölt, amelynek törzsében az oszlophivatkozás a lambda paraméterével prefixelve szerepel.

```
// SQL: UPDATE Users SET Role = 'Admin' WHERE Age > 30
// Generált LINQ:
db.Users.Where(x => x.Age > 30).ExecuteUpdateAsync(s => s
    .SetProperty(x => x.Role, "Admin"))

// SQL: UPDATE Users SET Points = Points + 10, Bonus = 0
   WHERE Role = 'User'
// Generált LINQ:
db.Users.Where(x => x.Role == "User").ExecuteUpdateAsync(
   s => s
    .SetProperty(x => x.Points, x => x.Points + 10)
    .SetProperty(x => x.Bonus, 0))
```

18. ábra. UPDATE fordítása konstans és kifejezés értékkel

A `NULL` értékre való frissítés külön kezelést igényel. A `SetProperty(x => x.Bonus, null)` alak fordítási hibát okozna, mivel a fordítóprogram nem tudja inferálni a `null` típusát. A megoldás lambda alak alkalmazása: `SetProperty(x => x.Bonus, x => null)`, ahol az EF Core a bal oldali lambda visszatérési típusából (`int?`) következtet ki a `null` típusát.

A `SET` kifejezések látogatása ugyanazzal a `_inWhereClause` flag mechanizmussal történik, mint a WHERE feltételeknél, így az oszlophivatkozások (`Points`, `Role`) helyesen kapják meg az `x.` prefixet a generált kódban.

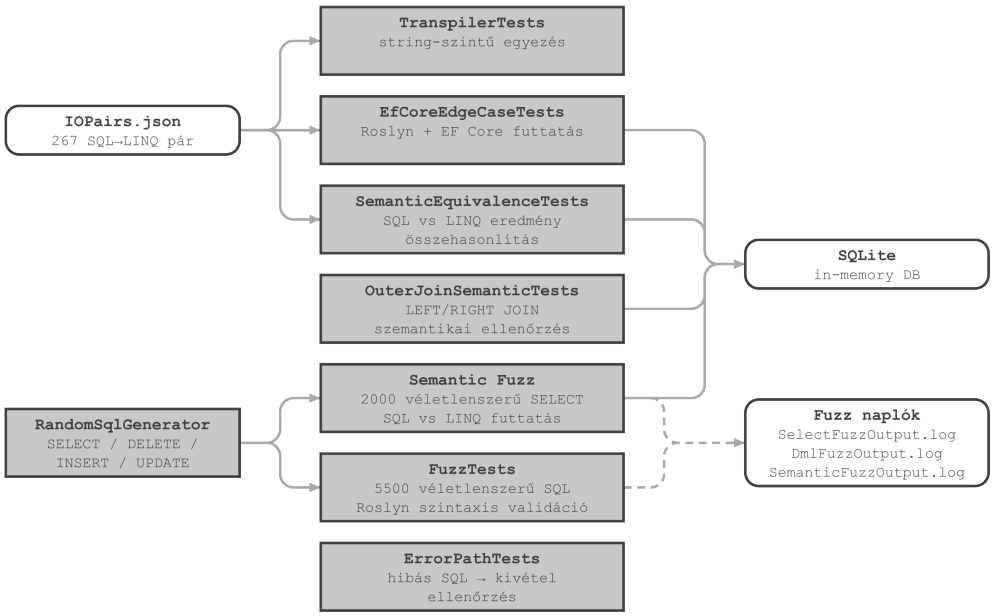
5. Tesztelés

A fordító helyességének igazolása három egymást kiegészítő tesztelési szinten történik. Mindhárom szint más-más típusú hibát képes felderíteni, és együttesen lefedik a fordítás teljes folyamatát a szintaktikai elemzéstől a szemantikai egyenértékűségig. A tesztelési infrastruktúra felépítését a 19. ábra mutatja.

A tesztosztályok és a bennük található tesztek számát a 2. táblázat foglalja össze.

Tesztosztály	**Szint**	**Tesztek**	**Megjegyzés**
`TranspilerTests`	string-szintű	267	`IOPairs.json` alapján
`ErrorPathTests`	string-szintű	30	hibaágak, kivételtípusok
`PascalCaseTests`	string-szintű	5	névkonverzió
`SemanticEquivalenceTests`	szemantikai	267 + 1	190 futó, 77 kizárt eset, 1 fuzz teszt
`EfCoreEdgeCaseTests`	szemantikai	80	EF Core fordítás, DML
`OuterJoinSemanticTests`	szemantikai	13	outer join, külön séma
`FuzzTests`	fuzz	2	2500 `SELECT` és 3000 DML iteráció

2. táblázat. A tesztosztályok összesítése



19. ábra. A tesztelési infrastruktúra felépítése

5.1. Transpiler tesztek

A legegyszerűbb és leggyorsabb tesztelési szint az `IOPairs.json` fájlban tárolt SQL→LINQ párok string-szintű összehasonlítása. A fájl 267 tesztesetet tartalmaz, amelyek minden támogatott SQL konstrukciót lefednek, az egyszerű `SELECT` lekérdezésektől a háromtáblás JOIN-okon, korrelált allekérdezéseken és `GROUP BY` aggregátumokon át a DML utasításokig.

A `TranspilerTests` osztály minden tesztesethez meghívja a `SqlToLinqConverter.Convert()` metódust, majd a generált és az elvárt LINQ kódot szóközök eltávolítása után hasonlítja össze. Ez a normalizálás szükséges, mert a fordító nem garantál egzakt whitespace-formázást, csak a kód szerkezetét. Ha az összehasonlítás sikertelen, a hibaüzenet tartalmazza a bemeneti SQL-t, a generált és az elvárt kimenetet, megkönnyítve a regresszió azonosítását.

A tesztek egy különálló `ErrorPathTests` osztályba is szerveződnek, amelyek azt ellenőrzik, hogy a fordító a megfelelő kivételt dobja érvénytelen vagy nem támogatott bemenet esetén. Ez magában foglalja a lexer- és parser-szintű szintaktikai hibákat (`SqlSyntaxException`), a nem fordítható SQL konstrukciókat (`NotSupportedException`), az ismeretlen függvényneveket, a helytelen argumentumszámot és a típusrendszerből fakadó korlátokat, például a string típusú oszlopokra alkalmazott összehasonlító operátorokat.

5.2. Szemantikai ekvivalencia tesztek

A string-szintű egyezés szükséges, de nem elégséges feltétele a helyességnek. Egy generált LINQ lekérdezés lehet szintaktikailag helyes, mégis különböző eredményt adhat mint az eredeti SQL. A `SemanticEquivalenceTests` osztály ezért egy lépéssel tovább megy, mind a bemeneti SQL-t, mind a belőle generált LINQ kódot ténylegesen lefuttatja egy SQLite in-memory adatbázison, és az eredményhalmazokat összehasonlítja.

A végrehajtás a Microsoft Roslyn fordítóplatform segítségével történik. A generált LINQ kódot a tesztelési infrastruktúra dinamikusan C# forráskóddá formázza, majd a `CSharpCompilation` osztállyal memóriában lefordítja és futtatja. Ez a megközelítés lehetővé teszi, hogy a tesztek ne csak a fordító kimenetét ellenőrizzék, hanem annak tényleges futási eredményét is, mindenféle kézzel írt ragasztókód nélkül. Ha a Roslyn fordítás sikertelen, a hibaüzenet tartalmazza a generált forráskódot és az összes fordítási hibát, ami megkönnyíti a diagnosztikát.

Az adatbázis egy rögzített, ismételhető seed adatokkal van feltöltve. A seed adatok gondosan megtervezettek, minden táblában vannak olyan sorok, amelyek egymással join-kapcsolatban állnak, és olyanok is, amelyek nem, ez teszi lehetővé az outer join szemantika helyes ellenőrzését. Az eredményösszehasonlítás sor szintű, az ORDER BY záradék nélküli lekérdezéseknél a sorok sorba rendezés után, rendezéssel rendelkezőknél eredeti sorrendben kerülnek összevetésre.

Bizonyos teszteseteket ki kellett zárni ebből a szintből. A 267 esetből 80 nem fut le szemantikai összevetéssel, a kizárásnak pedig három fő oka van. Az első a dialektusfüggő függvények köre. Az SQLite nem tartalmaz minden SQL Server, MySQL vagy Oracle specifikus függvényt, így a `YEAR()`, `MONTH()`, `DATEADD()`, `DATEDIFF()`, `GETDATE()` és hasonló dátumfüggvények, valamint a `LEN()`, `ISNULL()`, `NVL()`, `STUFF()` és rokon szöveg- és matematikai függvények nyers SQL-ként nem futtathatók. A második ok a logikai literálok eltérő kezelése, mivel az SQLite a `TRUE` és `FALSE` értékeket egész számként adja vissza, míg a generált LINQ `bool` típusú eredményt állít elő. A harmadik csoportot a `DELETE`, `INSERT` és `UPDATE` utasítások alkotják (összesen 28 eset), amelyek nem eredményhalmazt adnak vissza, hanem az adatbázis állapotát módosítják, ezért ezek ellenőrzése külön, az `EfCoreEdgeCaseTests` osztályban történik. A kizárt eseteket egy explicit kizárási lista alapján az `Assert.Ignore()` hívással átugorja a rendszer, a helyességüket pedig a transpiler tesztek biztosítják. A fennmaradó 187 eset esetében a nyers SQL és a generált LINQ eredményhalmaza ténylegesen összevetésre kerül.

A `DELETE`, `INSERT` és `UPDATE` utasítások szemantikai ellenőrzése külön `EfCoreEdgeCaseTests` osztályban történik. Ezek a tesztek ténylegesen mó-

dosítják az adatbázist, majd az EF Core `DbContext`-en keresztül ellenőrzi az adatbázis állapotát. Mivel a módosítások megváltoztatják a seed adatokat, minden teszt esetén a `[SetUp]` metódus újra feltölti az adatbázist. A `NULL` értékre való `UPDATE` esetén külön figyelmet igényel az EF Core type inference, mivel a `SetProperty(x => x.Col, null)` alak fordítási hibát okoz, mert a fordítóprogram nem tudja inferálni a `null` típusát, ezért a generált kód `SetProperty(x => x.Col, x => null)` alakot használ, ahol a bal oldali lambda visszatérési típusából az EF Core következteti ki a megfelelő nullable típust.

Az outer join szemantika tesztelése egy külön `OuterJoinSemanticTests` osztályban történik, dedikált `OuterJoinDbContext`-tel és seed adatokkal. A seed adatok szándékosan tartalmaznak illeszkedés nélküli sorokat mindkét oldalon, a `Customers` táblában vannak rendelés nélküli vásárlók, az `Invoices` táblában pedig nem létező vásárlóra hivatkozó számlák. Ez teszi lehetővé, hogy a `LEFT JOIN` által megőrzött bal oldali és a `RIGHT JOIN` által megőrzött jobb oldali sorok viselkedése egyaránt ellenőrizhető legyen.

5.3. Fuzz tesztek

A fuzz tesztelés a fordító robusztusságát vizsgálja véletlenszerűen generált, de grammatikailag érvényes SQL lekérdezéseken. A `RandomSqlGenerator` osztály egy rögzített maggal (seed) inicializált véletlenszám-generátor segítségével állít elő SQL utasításokat, amelyek lefedik a fordító összes főbb ágát.

A `FuzzTests` osztály két tesztet tartalmaz. Az első 2500 véletlenszerű `SELECT` lekérdezést fordít le és ellenőrzi a generált C# kód szintaktikai helyességét a Roslyn `SyntaxFactory.ParseExpression()` metódusával. A második 3000 DML utasítást generál egyenlő arányban `DELETE`, `INSERT` és `UPDATE` között, és a generált kódot `CSharpSyntaxTree.ParseText()` segítségével validálja, mivel a DML kimenet statement, nem kifejezés. Mindkét teszt logfájlba írja az eredményeket (`SelectFuzzOutput.log`, `DmlFuzzOutput.log`), amelyek `[PASS]`, `[SKIP]` és `[FAIL]` bejegyzéseket tartalmaznak.

A `NotSupportedException`-nel záruló eseteket a fuzz tesztek `[SKIP]`-ként kezelik, ezek olyan véletlenszerűen generált kombinációk, amelyek egyébként érvényes SQL-t alkotnak, de a fordító nem támogatja őket (például string típusú oszlopra alkalmazott `BETWEEN` operátor). Ezt a viselkedést az `ErrorPathTests` osztály explicit tesztesetei ellenőrzik.

A `SemanticEquivalenceTests` osztály szintén tartalmaz egy fuzz alapú szemantikai tesztet, amely 2000 véletlenszerű `SELECT` lekérdezést fordít le, majd mind a nyers SQL-t, mind a generált LINQ-ot lefuttatja az SQLite adatbázison, és az eredményhalmazokat összehasonlítja. Ez a teszt a legkörülményesebb, de egyben a legerősebb biztosíték, mivel ha itt sikeres, az

azt jelenti, hogy a fordítás nem csupán szintaktikailag helyes kódot állít elő, hanem szemantikailag ekvivalens lekérdezést is.

6. LLM összehasonlítás

A projekt egyik motivációja az volt, hogy a nagy nyelvi modellek (LLM) ugyan képesek SQL lekérdezéseket LINQ kóddá alakítani, de ez a folyamat statisztikai alapú, nem determinisztikus, és nem integrálható automatizált pipeline-ba. Ennek empirikus vizsgálatára méréseket végeztem, amelyek során két kereskedelmi LLM-et hasonlítottam össze a saját fordítóval ugyanazon a bemeneten.

6.1. Módszertan

A mérés bemenete a projekt stressz tesztelésére összeállított komplex SQL lekérdezés, amely egyszerre tartalmaz háromtáblás JOIN-t, beágyazott `CASE WHEN` kifejezést, korrelált és nem korrelált allekérdezéseket, `EXISTS` és `NOT IN` feltételeket, szöveg- és dátumfüggvényeket, `COALESCE`-t, `CAST`-ot, valamint `ORDER BY`, `LIMIT` és `OFFSET` záradékokat. A lekérdezés szándékosan összetett, hogy megvizsgálja, hogyan birkóznak meg az LLM-ek a grammatikailag és szemantikailag bonyolult bemenetekkel.

Minden modell ugyanazt a promptot kapta, amely kizárólag a LINQ kifejezést kérte vissza, magyarázat és formázás nélkül. A mérést mindkét modellnél 5 alkalommal végeztem el, és rögzítettem a válaszidőt, a kimenet tartalmát, valamint azt, hogy a különböző futtatások azonos kimenetet adtak-e.

A két vizsgált LLM a Gemini 3.6 Flash (Google) és a GPT-OSS-120B (OpenAI, Groq infrastruktúrán futtatva) volt. A saját fordító mérései egy fejlesztői gépen, lokálisan futtatva történtek, ahol az első hívás utáni JIT warm-up időt nem vettük figyelembe. Fontos megjegyezni, hogy a válaszidőn kívül az erőforrásfelhasználásban is nagyságrendi különbségek állnak fenn: az LLM-ek esetén nagyteljesítményű felhőinfrastruktúra fut a háttérben, míg a saját fordító egy átlagos fejlesztői gépen is ezredmásodpercek alatt elvégzi a feladatot.

Az összehasonlítás alapját a generált kimenetek karakterszintű összevetése adja, kiegészítve strukturális elemzéssel. Karakterhelyes egyezés az LLM-ek esetén nem várható, mivel a modellek ugyanarra a bemenetre különböző, de esetleg szemantikailag egyenértékű kódot generálhatnak. Ezért a konzisztenciát nem karakterszintű azonosságként értelmezzük, hanem azt vizsgáljuk, hogy a különböző futtatások strukturálisan hasonló vagy eltérő megoldásokat adnak-e.

A mérések technikai megvalósítása egy Python szkripten alapul, amely az adott modell nyilvános API-ját hívja, és `time.perf_counter()` segítségével rögzíti a kérés elküldésétől a teljes válasz megérkezéséig eltelt időt. Az 5 futtatás mindegyikénél azonos API-paramétereket alkalmaztam, 0 hőmérséklet értékkel és rögzített maximális tokenszámmal, hogy a variancia forrása a modell belső mintavételezésében legyen keresendő, ne a konfigurációban. A saját fordító futásidejét ugyanabban a szkriptben, az `Environment.TickCount64` alapú .NET beépített stopperrel mértem.

Fontos rendszerszintű tényező, hogy az LLM-mérések hálózati kéréseket tartalmaznak. A rögzített válaszidő magában foglalja a DNS feloldást, a TLS kézfogást, a szerver oldali feldolgozást és a streamelt válasz fogadásának teljes idejét, így a mért értékek nem csupán a modell sebességét tükrözik, hanem a hálózati infrastruktúra pillanatnyi állapotát is. Ez részben magyarázza a magas szórást. A saját fordító lokálisan fut, hálózati késleltetés nélkül, ezért a két mérési környezet nem közvetlen sebességként hasonlítandó össze, hanem inkább a pipeline-ba illeszthetőség szempontjából érdemes értékelni őket.

6.2. Eredmények

Modell	**Átlag**	**Min**	**Max**	**Szórás**	**Konzisztens**
Gemini 3.6 Flash	46,18 s	17,43 s	71,30 s	22,1 s	nem
GPT-OSS-120B (Groq)	13,82 s	3,43 s	20,02 s	6,8 s	nem
Saját fordító (lokális)	$<$1 ms	$<$1 ms	$<$1 ms	$\approx$0	igen

3. táblázat. LLM összehasonlítás eredményei (5 futtatás)

A Gemini esetén a válaszidők szórása 22,1 másodperc, ami a minimális és maximális értékek közötti közel négyszeres különbséget tükröz. A Groq modellnél a szórás 6,8 másodperc, a legrövidebb és leghosszabb válasz között közel hatszoros eltérés mutatkozik. Ez a szórás önmagában is jelzi, hogy az LLM-ek válaszideje kiszámíthatatlan, ami automatizált pipeline-ban tervezési nehézséget jelent.

6.3. Megfigyelések

A válaszidő tekintetében a különbség több nagyságrendnyi. A Gemini átlagosan 46 másodpercet vett igénybe ugyanahhoz a lekérdezéshez, amelyet a saját fordító ezredmásodpercek alatt dolgoz fel lokálisan. A Groq infrastruktúrán futó GPT-OSS-120B gyorsabbnak bizonyult, de a 13 másodperces átlag

és a 20 másodperces maximum még mindig lényegesen több, mint ami egy automatizált pipeline-ban elfogadható lenne.

A konzisztencia mindkét LLM esetén hiányzott. A Gemini 5 futtatásán 5 strukturálisan különböző kimenetet produkált. A Groq modell szintén eltérő megoldásokat adott futtatásonként, emellett két futtatás üres választ, egy pedig csonka, félbeszakadt kimenetet adott vissza. A saját fordító mind az 5 futtatáson byte-szintűen azonos kimenetet állított elő.

A kimenet helyességét vizsgálva szembetűnő strukturális hibák is megfigyelhetők. A Groq modell első válaszában a `WHERE` feltétel a `JOIN` műveletek elé kerül, ami az EF Core lekérdezési láncban szemantikailag helytelen sorrendet jelent. A `DbContext` elnevezése futtatásonként változott (`ctx`, `context`, `db`), és a `LIKE 'B%'` operátor egyszer `EF.Functions.Like`-ra, másszor `StartsWith()`-re fordult. A saját fordító mindkét esetben providerfüggetlen, EF Core kompatibilis, determinisztikus kimenetet állít elő.

6.4. Következtetések

Az LLM-ek interaktív segítségként hasznos eszközök lehetnek, ha egy fejlesztő egyszeri alkalommal szeretne egy SQL lekérdezést LINQ-ra fordítani és az eredményt manuálisan ellenőrzi. Automatizált migrációs pipeline-ban azonban, ahol több száz vagy több ezer lekérdezést kell feldolgozni, a válaszidő, a nem determinisztikus kimenet, az ellenőrizhetőség hiánya és a felhőinfrastruktúra szükségessége komoly akadályt jelent. A determinisztikus fordító ezzel szemben programozottan hívható, az eredménye verifikálható, lokálisan futtatható, és ugyanarra a bemenetre mindig ugyanazt adja.

Az összehasonlítás egy mélyebb architekturális különbséget is feltár, amely a két megközelítés természetéből következik. Az LLM-ek fekete dobozként működnek, azaz a bemenetre adott válasz belső folyamata nem figyelhető meg, nem ellenőrizhető és nem reprodukálható megbízhatóan. Két strukturálisan azonos SQL lekérdezés esetén a modell más szintaktikai szerkezetet választhat, más segédváltozó-nevet generálhat, és más EF Core metódust alkalmazhat, miközben mindkét kimenet szemantikailag helyes is lehet. Ez interaktív, emberi ellenőrzés melletti használatban elfogadható, de automatizált környezetben azt jelenti, hogy a kimenet minden futtatás után felülvizsgálatot igényel.

A determinisztikus fordító ezzel szemben fehér dobozként működik, ahol minden bemeneti tokenhez pontosan meghatározott fordítási szabály tartozik. Az AST csomópontjai egy az egyben megfeleltethetők a generált kód részleteinek, és a teljes fordítási folyamat visszakövethető. Ha a kimenet hibás, a hiba lokalizálható és javítható anélkül, hogy az egész rendszert újra kellene tanítani. Ez a verifikálhatóság teszi a determinisztikus fordítót alkalmassá

olyan feladatokra, ahol az eredmény helyessége nem opcionális.

7. A kódgenerálás leválasztása

Az előző fejezetekben bemutatott fordító egy kész, tesztelt eszköz, amely SQL utasításokból EF Core kompatibilis LINQ kódot állít elő. A fejlesztés lezárása után felmerült annak a lehetősége, hogy a fordító ne csak egyetlen célnyelvet szolgáljon ki. A grammatika, a parser és a visitor által felépített köztes fa ugyanis nem kötődik szükségszerűen a C#-hoz, hiszen az SQL lekérdezés szerkezetét írják le. Ha a kódgenerálás cserélhető elemmé válik, ugyanaz a fordító SQL utasításokból tetszőleges célnyelvre képes kódot előállítani, például Java Stream API hívásokra. Ebben a fejezetben ennek az átalakításnak a lehetséges megközelítéseit, a választott megoldást és annak korlátait mutatom be.

7.1. Motiváció és kiinduló állapot

Az eredeti tervben a kódgenerálás a köztes fa csomópontjainak feladata volt. Minden csomóponttípus saját `ToCodeString()` metódussal rendelkezett, amely közvetlenül C# kódot állított elő. Egy második célnyelv támogatásához tehát első ránézésre elegendő lett volna ezeket a metódusokat egy cserélhető komponensbe kiszervezni.

A kód részletesebb vizsgálata azonban megmutatta, hogy a C# szintaxis nem csak a csomópontokban, hanem a visitor rétegben is jelen volt. A visitor több helyen kész C# szöveget írt a köztes fába. Az oszlophivatkozások `LinqIdentifierNode` csomópontként, stringként összerakott előtagokkal kerültek a fába (`g.Key`, `g.FirstOrDefault().Oszlop`, `x.u.Oszlop`). A szöveg-összefűzés `string.Concat(...)`, a `COUNT(DISTINCT ...)` pedig egy teljes `g.Select(...).Distinct().Count()` kifejezés formájában jelent meg. A `GETUTCDATE()` és a `RAND()` függvények eredménye a `DateTime.UtcNow` és a `new Random().NextDouble()` szöveg lett, az inline nézetek belső lekérdezése pedig már generált kódként került a fába. Az operátorok (`==`, `&&`, `!`) és a `CAST` céltípusai (`int`, `DateTime`) szintén C# jelölésként szerepeltek. Egy másik célnyelvhez tartozó generátor ezeket a szövegeket már nem tudta volna átalakítani, ezért a kódgenerálás leválasztásához a köztes fát is célnyelvfüggetlenné kellett tenni.

7.2. Lehetséges megközelítések

A cserélhető kódgenerálás megvalósítására három megközelítést vizsgáltam meg. Ezek abban különböznek, hogy a célnyelvi szabályok hol és milyen formában kerülnek leírásra.

7.2.1. XSLT alapú transzformáció

Az XSLT (Extensible Stylesheet Language Transformations) a W3C által szabványosított, deklaratív nyelv XML dokumentumok átalakítására [3]. Egy XSLT stíluslap sablonszabályokból áll, amelyek a bemeneti fa csomópontjaira illeszkednek, és meghatározzák, hogy az adott csomópontból milyen kimenet keletkezzen. A kimenet nem csak XML lehet, hanem tetszőleges szöveg is, így az XSLT forráskód generálására is alkalmas.

Ebben a megközelítésben a köztes fát először XML formátumba kellene szerializálni, ahol minden csomóponttípus egy XML elemnek, a csomópont tulajdonságai attribútumoknak, a gyermekcsomópontok pedig beágyazott elemeknek felelnének meg. Az egyes célnyelveket egy-egy stíluslap írná le. A 20. kódrészlet egy ilyen stíluslap két sablonszabályát mutatja be, amelyek egy lambda kifejezést és egy tagelérést alakítanak C# kóddá.

```
<xsl:template match="Lambda">
  <xsl:value-of select="@parameter"/>
  <xsl:text> => </xsl:text>
  <xsl:apply-templates select="*"/>
</xsl:template>

<xsl:template match="MemberAccess">
  <xsl:apply-templates select="*"/>
  <xsl:text>.</xsl:text>
  <xsl:value-of select="@member"/>
</xsl:template>
```

20. ábra. XSLT sablonszabályok egy lambda kifejezés és egy tagelérés generálására

A megközelítés legnagyobb előnye, hogy egy új célnyelv támogatása a fordító újrafordítása nélkül, egy új stíluslap hozzáadásával megoldható. A mintaillesztésen alapuló sablonszabályok természetes módon követik a fa szerkezetét, és a szabvány érettsége miatt az XSLT jól dokumentált, széles körben ismert technológia.

A projekt szempontjából azonban jelentős hátrányai is vannak. A .NET beépített `XslCompiledTransform` osztálya csak az XSLT 1.0 ajánlást támogatja [11], amely szövegkezelésben és feltételes logikában erősen korlátozott. A kódgenerálás több pontja ráadásul nem írható le egyszerű, csomópontonkénti sablonnal. A `LEFT JOIN` eredményvetítésének átírása, a skaláris allekérdezések felismerése vagy a dátumfüggvények egységeinek feldolgozása mind olyan döntés, amely a csomópont környezetétől vagy a gyermekekből már előállított kódtól függ. Ezek XSLT 1.0-ban csak nehezen követhető, rekurzív sablonokkal valósíthatók meg. Az átalakítás minden hibája csak futásidőben derülne ki, a stíluslap hibakeresése körülményes, és a köztes fa minden csomóponttípusához XML szerializációt is meg kellene valósítani. A szükséges ráfordítás így a projekt kereteihez képest aránytalanul nagy lett volna.

7.2.2. Deklaratív szabályrendszer

Egy másik lehetőség a célnyelvi szabályok JSON vagy YAML formátumú leírása. Egy ilyen szabályrendszer csomóponttípusonként, operátoronként és függvényenként egy-egy helyőrzőket tartalmazó sablont rendelne a célnyelvhez, például a `SUBSTRING` függvényhez a `{0}.Substring({1} - 1, {2})` sablont. Ez a megoldás az egyszerű, egy az egyben megfeleltethető elemeknél (operátorok, skaláris függvények) kényelmes és könnyen bővíthető. A környezetfüggő döntések leírásához viszont a szabályrendszert feltételekkel és saját kifejezésnyelvvel kellene kiegészíteni, ami lényegében egy új sablonnyelv megalkotását jelentené. A megközelítés ezért a választott megoldás kiegészítéseként lehet érdekes, például a függvényleképezések külső fájlba szervezésére.

7.2.3. Kódgenerátor interfész

A harmadik megközelítésben a célnyelvi szabályokat egy általános célú programozási nyelven megírt osztály tartalmazza, amely egy rögzített interfészt valósít meg. Ennek előnye, hogy a generátor írója a nyelv teljes kifejezőerejét használhatja, a fordítóprogram ellenőrzi a kódját, és a környezetfüggő döntések is közvetlenül megfogalmazhatók. Hátránya, hogy egy új célnyelv támogatásához programozni kell, és a generátort a fordítóval együtt kell lefordítani. A projekt céljaihoz ez a megközelítés illeszkedik a legjobban, mivel a meglévő `ToCodeString()` metódusok logikája változtatás nélkül átvihető, és a megoldás a meglévő tesztkészlettel teljes körűen ellenőrizhető.

7.3. Az ICodeGenerator megvalósítása

Az átalakítás után a fordítás folyamata egy új lépéssel bővült. A visitor továbbra is a köztes fát építi fel, a fából azonban már nem a csomópontok, hanem egy külön kódgenerátor állítja elő a célnyelv kódját. A teljes folyamat tehát a parse fa, a `SqlVisitor`, a köztes fa, a kódgenerátor és a kimeneti kód láncaként írható le.

A kódgenerátor interfésze a visitor tervezési mintát követi [6]. A `ILinqNodeVisitor<T>` interfész a köztes fa minden csomóponttípusához pontosan egy `Visit` metódust ír elő, összesen harmincat. Az `ICodeGenerator` ennek a `string` visszatérési típusú változata, amely a `Generate` metódussal egy belépési pontot is biztosít. Minden csomópont egy egysoros `Accept` metódust kapott, amely a generátor megfelelő `Visit` metódusát hívja meg (21. kódrészlet).

```
public interface ICodeGenerator : ILinqNodeVisitor<string
    > {
    string Generate(LinqNode root);
}

public interface ILinqNodeVisitor<out T> {
    T VisitQuery(LinqQueryNode node);
    T VisitMethodCall(LinqMethodCallNode node);
    T VisitLambda(LinqLambdaNode node);
    // ... csomóponttípusonként egy metódus
}

public class LinqLambdaNode : LinqNode {
    public string ParameterName { get; set; }
    public LinqNode Body { get; set; }

    public override T Accept<T>(ILinqNodeVisitor<T>
        visitor)
         => visitor.VisitLambda(this);
}
```

21. ábra. A kódgenerátor interfésze és egy csomópont Accept metódusa

A visitor minta választásának több oka is volt. Az interfész maga írja le, hogy egy új generátornak mely csomóponttípusokat kell kezelnie, így az a generátor írójának egyben kitöltendő metóduslistaként is szolgál. Ha

a köztes fába új csomóponttípus kerül, minden meglévő generátor fordítási hibát ad mindaddig, amíg az új típus kezelése el nem készül, így egyetlen csomóponttípus kezelése sem maradhat ki észrevétlenül. A minta ráadásul a fordító bemeneti oldalán is jelen van, hiszen az ANTLR által generált `SqlParserBaseVisitor` szintén erre épül. A fordító így mindkét irányban ugyanazt a bejárási elvet követi.

A C# kód előállítását a `CSharpCodeGenerator` osztály végzi, amelybe a korábbi `ToCodeString()` metódusok logikája változatlan formában került át. A fordító teljes C# specifikus szintaxisa ebben az egy osztályban található. Az osztály üres sorok és megjegyzések nélkül nagyjából 480 sor, amelynek mintegy harmadát a skaláris SQL függvények leképezése teszi ki. Egy új célnyelvhez tartozó generátor a csomóponttípusok kezelését így nagyjából 200 és 300 sor közötti kóddal valósíthatja meg, amihez a támogatni kívánt függvénykör leképezése adódik hozzá. A `SqlToLinqConverter` osztály a meglévő hívások mellett egy olyan változatot is kapott, amely paraméterként tetszőleges `ICodeGenerator` implementációt fogad, generátor megadása nélkül pedig továbbra is a C# generátort használja.

A köztes fa célnyelv-függetlenné tételéhez a visitor által előállított C# szövegeket új csomóponttípusok váltották fel. A `LinqIdentifierNode` megszűnt, helyét a lambda paraméterre hivatkozó `LinqParameterNode`, a tagelérést leíró `LinqMemberAccessNode`, valamint a csoportkulcsot (`LinqGroupKeyNode`) és a csoport egy elemét (`LinqGroupElementNode`) reprezentáló csomópontok vették át. Az operátorok és a `CAST` céltípusai felsorolás típusként kerülnek a fába, a szöveg-összefűzés és a `RAND()` a skaláris függvények közös csomópontjára képződik le, a `COUNT(DISTINCT ...)` pedig az aggregátum csomópont egy jelzőjeként jelenik meg. Az inline nézetek a belső lekérdezés kódja helyett annak részfáját tárolják.

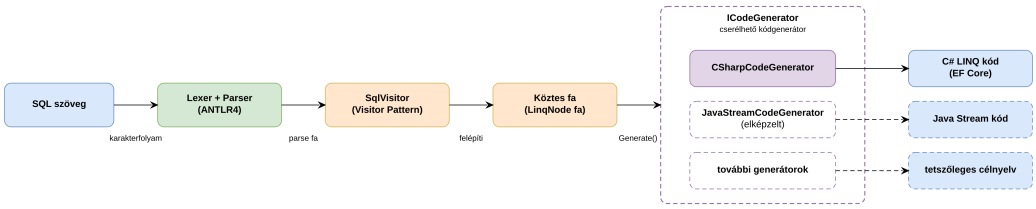
Egy tervezési kérdést tudatosan pragmatikus módon oldottam meg. A köztes fa szerkezete és a metódusnevek (`Where`, `Select`, `OrderBy`, `GroupBy`) továbbra is a LINQ műveleti modelljét követik, mivel a fordító elsődleges célnyelve a LINQ. Ezek a nevek a fában kanonikus műveletnévként szerepelnek, amelyeket egy másik célnyelv generátora a saját szókészletére fordíthat le. A 22. kódrészlet azt mutatja be, hogyan képezhetné le egy Java Stream API-t előállító generátor a metódusneveket.

```
public string VisitMethodCall(LinqMethodCallNode node) {
    string name = node.MethodName switch {
        "Where"   => "filter",
        "Select"  => "map",
        "OrderBy" => "sorted",
        "Take"    => "limit",
        "Skip"    => "skip",
        "ToList"  => "toList",
        _ => throw new NotSupportedException(node.
           MethodName)
    };
    var args = string.Join(", ", node.Arguments.Select(
       Gen));
    return $".{name}({args})";
}
```

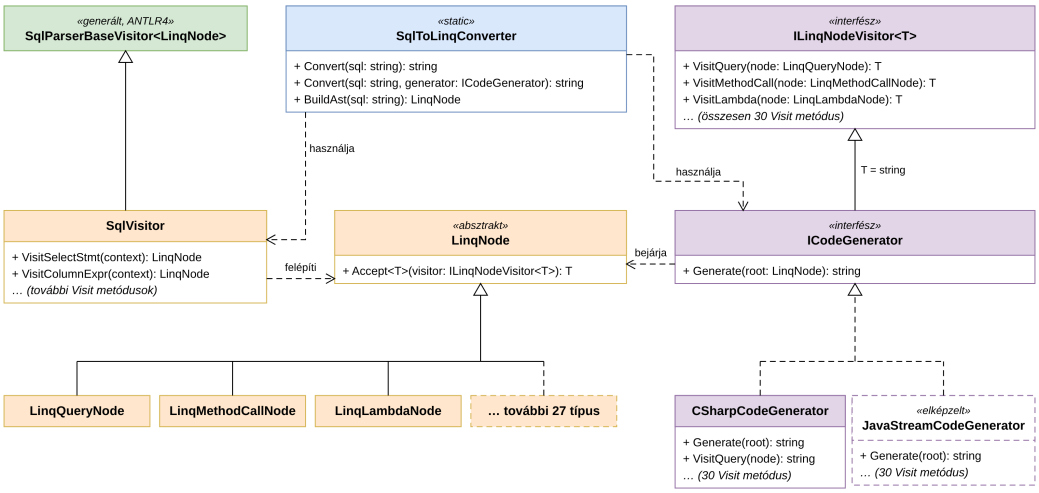
22. ábra. Metódusnevek leképezése egy elképzelt Java Stream generátorban

Egy teljesen absztrakt, műveletfajtákat felsorolásként tároló fa a célnyelvektől még függetlenebb lenne, de lényegesen nagyobb átalakítást igényelt volna, és a C# generátor esetében semmilyen előnnyel nem járt volna.

A fordítás új felépítését a 23. ábra, a kódgenerálásban részt vevő osztályok kapcsolatát a 24. ábra mutatja.



23. ábra. A fordítás folyamata a cserélhető kódgenerátorral



24. ábra. A kódgenerálás osztálydiagramja

7.4. Az átalakítás ellenőrzése

Az átalakítás során a fordító viselkedésének változatlannak kellett maradnia. Ennek ellenőrzésére a korábban bemutatott tesztkészlet szolgált. A 267 transpiler teszteset elvárt kimenetén nem kellett módosítani, és az átalakítás után a szemantikai, az EF Core specifikus és a fuzz tesztek is hiba nélkül lefutottak. A string-szintű regressziós tesztek így nem csak a fordító helyességét igazolták, hanem lehetővé tették egy jelentős belső átalakítás biztonságos elvégzését is.

7.5. Korlátok

A kódgenerálás leválasztása a szintaktikai különbségeket kezeli, a célnyelvek közötti szemantikai eltéréseket azonban nem. A LINQ és az EF Core legfontosabb tulajdonsága, hogy a lekérdezés kifejezésfaként jut el a providerhez, amely azt SQL-re fordítja és az adatbázisban hajtja végre. A Java Stream API ezzel szemben a memóriában lévő gyűjteményeken dolgozik, így egy Stream alapú generátor kimenete szintaktikailag helyes lehet, a lekérdezés mégis a teljes tábla betöltése után, kliensoldalon futna le. Hasonló eltérés a null-értékek kezelése, mivel a C# nullable típusai és a `??` operátor nem minden nyelvben léteznek, a többoszlopos vetítéshez használt névtelen típusok helyett pedig sok nyelvben előre deklarált rekordtípusokra van szükség. A köztes fa LINQ-alakú szerkezete miatt végül egy, a LINQ műveleti modelljétől lényegesen eltérő célnyelv (például egy tisztán deklaratív lekérdezőnyelv) támogatása a fa szerkezetének módosítását is igényelheti. Ezek a kérdések

egy konkrét második generátor megvalósításakor válnak eldönthetővé, és további fejlesztési irányt jelölnek ki.

8. Összegzés

A jelen munka egy SQL:1999 kompatibilis lekérdezéseket EF Core 8.0 LINQ metódusszintaxisra fordító eszköz tervezését és megvalósítását mutatta be. A projekt két alapvető kérdést vizsgált: milyen SQL nyelvi elemek fordíthatók egyértelműen LINQ-ra, és milyen architektúra teszi lehetővé egy ilyen fordító megbízható és bővíthető megvalósítását.

Az első kérdésre a válasz a projekt hatókörén keresztül rajzolódott ki. A `SELECT` utasítás teljes egészében leképezhető: a `WHERE` feltételek, a `JOIN` műveletek, a `GROUP BY` aggregáció, a halmazműveletek, az allekérdezések és a `CASE WHEN` kifejezések mind rendelkeznek egyértelmű LINQ megfelelővel. A `DELETE`, `INSERT` és `UPDATE` utasítások szintén fordíthatók, bár az EF Core 8.0 bulk-műveletei (`ExecuteDeleteAsync`, `ExecuteUpdateAsync`) és a change tracker alapú műveletek (`Add`, `SaveChanges`) közötti különbség tudatos tervezési döntést igényelt. Bizonyos SQL konstrukciók viszont nem fordíthatók le megbízhatóan: a `FULL OUTER JOIN`, a közös táblakifejezések (CTE) és az ablakfüggvények nem rendelkeznek olyan LINQ megfelelővel, amelyet az EF Core minden provideren szerver oldalon SQL-re fordítana.

A második kérdésre a háromrétegű architektúra adott választ. Az ANTLR4-alapú parser, a saját LINQ AST és a rekurzív kódgenerátor szétválasztása lehetővé tette, hogy minden réteg önállóan fejleszthető és tesztelhető legyen. Az AST réteg különösen fontos szerepet játszott, mivel a parse fa és a LINQ AST szerkezetének különbsége nem pusztán technikai részlet, hanem a fordítás lényege. Ahol a parse fa szélességében nagy és az SQL szintaxis minden elemét tartalmazza, a LINQ AST mélységében strukturált és csak azokat az információkat hordozza, amelyek a kódgeneráláshoz szükségesek. A rétegek szétválasztása később azt is lehetővé tette, hogy a kódgenerálás a fordító többi részének módosítása nélkül cserélhető komponenssé váljon.

A fejlesztés során felmerülő kihívások visszatérő mintát mutattak, az első, kézenfekvőnek tűnő megközelítés szinte minden esetben valamilyen szélső esetben megbukott. A `LIKE` operátor regex-alapú fordítása kliensoldalon ragadt, az `IN (SELECT ...)` `Contains()`-alapú implementációja nullable típusoknál típusillesztési hibát dobott, a scope kezelés első változata lambda névütközéseket okozott mélyen egymásba ágyazott allekérdezéseknél. Ezek a tapasztalatok rávilágítottak arra, hogy egy fordító helyessége nem igazolható pusztán a happy path eseteken végzett ellenőrzéssel, a regressziók megelőzése érdekében a tesztelési infrastruktúra legalább annyira fontos, mint maga a

fordítási logika.

A tesztelési megközelítés ennek megfelelően háromszintű lett. A string-szintű transpiler tesztek gyors visszajelzést adnak regressziókra. A Roslyn-alapú szemantikai tesztek igazolják, hogy a generált kód nem csupán szintaktikailag helyes, hanem valóban ugyanazt az eredményt adja, mint az eredeti SQL. A fuzz tesztek 5500 véletlenszerűen generált utasításon ellenőrzik a fordító robusztusságát olyan kombinációkban, amelyeket kézzel nehéz lenne megkonstruálni.

A projekt egy Blazor WebAssembly alapú webes demóval egészült ki, amely a fordítót interaktívan elérhetővé teszi a böngészőből, ANTLR parse fa és LINQ AST vizualizációval kiegészítve. Ez nemcsak demonstrációs célokat szolgál, hanem lehetővé teszi, hogy a fordító működése a parse fától a generált kódig vizuálisan is követhető legyen.

A kész fordító egy további lépésben keretrendszerré bővült. A kódgenerálás a köztes fa csomópontjaiból egy önálló, cserélhető komponensbe került, amelynek követelményeit az `ICodeGenerator` interfész rögzíti. Ehhez a köztes fát is célnyelv-függetlenné kellett tenni, mivel az eredeti megoldásban a visitor több helyen már a fa felépítésekor C# kódot állított elő. Az átalakítás után a C# kódot a `CSharpCodeGenerator` osztály állítja elő, egy másik célnyelv támogatásához pedig elegendő egy újabb generátor megírása, a grammatika, a parser és a visitor változtatás nélkül újrafelhasználható. Az átalakítást a meglévő tesztkészlet tette biztonságossá, mivel a 267 teszteset elvárt kimenetén nem kellett módosítani. A választott megoldás a célnyelvek közötti szintaktikai különbségeket kezeli. A szemantikai eltérések, például a szerveroldali kiértékelés hiánya a Java Stream API esetében, egy konkrét második generátor megvalósításakor válna kezelhetővé, ami a munka folytatását jelenti.

A munka egyik tanulsága, hogy az SQL és a LINQ közötti leképezés a felszínen egyszerűbbnek tűnik, mint amilyen valójában. Mindkét nyelv deklaratív absztrakciókat kínál az adatok felett, de az absztrakciók mögötti modellek, a relációs algebra és az objektumgráf lényegesen különböznek egymástól. Egy fordítóprogram nem kerülheti meg ezt a különbséget, csupán szisztematikusan kezelheti. A projekt rámutatott, hogy ez a szisztematikus kezelés megvalósítható, de minden egyes konstrukciónál meg kell érteni, hogyan viselkedik a célplatform, az EF Core query provider, és milyen korlátokat szab a fordított kódra.

Hivatkozások

[1] AngleSharp. AngleSharp: The ultimate angle brackets parser library.

https://github.com/AngleSharp/AngleSharp.

[2] Patrice Borne. Legacy database migration to the cloud — building a transpiler, 2021. https://medium.com/@patborne/legacy-database-migration-to-the-cloud-building-a-transpiler-4f673552f357.

[3] James Clark. XSL transformations (XSLT) version 1.0, 1999. W3C Recommendation, https://www.w3.org/TR/xslt-10/.

[4] F. Ferreira et al. nmorph framework: An innovative approach to transpiler-based multi-language software development. *ResearchGate*, 2023. https://www.researchgate.net/publication/375442986_nMorph_framework.

[5] Adam Freeman and Joseph C. Rattz Jr. *Pro LINQ: Language Integrated Query in C# 2010*. Apress, 2010.

[6] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[7] Nick Harrison. *Code Generation with Roslyn*. Apress, 2017.

[8] Christopher Ireland, David Bowers, Michael Newton, and Kevin Waugh. A classification of object-relational impedance mismatch. In *Proceedings of the First International Conference on Advances in Databases, Knowledge and Data Applications*. IEEE Computer Society, 2009.

[9] Erik Meijer, Brian Beckman, and Gavin Bierman. LINQ: Reconciling objects, relations and XML in the .NET framework. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*. ACM, 2006.

[10] Microsoft. Comparison of XPath and LINQ to XML. https://learn.microsoft.com/en-us/dotnet/standard/linq/comparison-xpath-linq-xml.

[11] Microsoft. XSLT transformations. https://learn.microsoft.com/en-us/dotnet/standard/data/xml/xslt-transformations.

[12] Microsoft. Complex query operators — EF core, 2023. https://learn.microsoft.com/en-us/ef/core/querying/complex-query-operators.

[13] MongoDB. Limitations — EF core provider. https://www.mongodb.com/docs/entity-framework/current/limitations/.

[14] Ted Neward. The vietnam of computer science, 2006. Elérhető: https://www.odbms.org/wp-content/uploads/2013/11/031.01-Neward-The-Vietnam-of-Computer-Science-June-2006.pdf.

[15] Francisco Ortin, Jose Quiroga, Oscar Rodriguez-Prieto, and Miguel Garcia. An empirical evaluation of Lex/Yacc and ANTLR parser generation tools. *PLOS ONE*, 17(3), 2022.

[16] Taylor Pacelli. MongoDB provider for entity framework core now generally available, 2024. https://www.mongodb.com/blog/post/mongodb-provider-entity-framework-core-now-generally-available.

[17] Michail Papaioannou and Christos Katsanos. Investigating the effects of object-relational impedance mismatch on the efficiency of object-relational mapping frameworks. *Journal of Database Management*, 31(4), 2020.

[18] Terence Parr. *Language Implementation Patterns*. Pragmatic Bookshelf, 2009.

[19] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2013.

[20] Jon Skeet. *C# in Depth, Fourth Edition*. Manning Publications, 2019.

[21] Manish Vasani. *Roslyn Cookbook: Compiler as a Service, Code Analysis, Code Quality and more*. Packt Publishing, 2017.